Laplacian Solvers

Generated by Doxygen 1.9.8

Chapter 1

README

1.1 challenge3

The third challenge

Chapter 2

results

=== Running Test 1: DIRICHLET SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 0 0 0 0 0 0 0 0 0 0.191446 0.344974 0.430175 0.430175 0.344974 0.191446 0 0 0.344974 0.621621 0.775148 0.775148 0.621621 0.344974 0 0 0.430175 0.775148 0.966594 0.966594 0.775148 0.430175 0 0 0.430175 0.775148 0.966594 0.966594 0.775148 0.430175 0 0 0.344974 0.621621 0.775148 0.775148 0.↩ 621621 0.344974 0 0 0.191446 0.344974 0.430175 0.430175 0.344974 0.191446 0 0 0 0 0 0 0 0 0 Residue error: 9.15183e-07 Total iterations: 115 Exact solution u_exact: 0 0 0 0 0 0 0 0 0 0.188255 0.339224 0.423005 0.423005 0.339224 0.188255 5.31354e-17 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩ 423005 0.762229 0.950484 0.950484 0.762229 0.423005 1.19394e-16 0 0.423005 0.762229 0.950484 0.950484 0.762229 0.423005 1.19394e-16 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩ 188255 0.339224 0.423005 0.423005 0.339224 0.188255 5.31354e-17 0 5.31354e-17 9.57467e-17 1.19394e-16 1.19394e-16 9.57467e-17 5.31354e-17 1.49976e-32

=== Running Test 2: NEUMANN SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1.01695 0.916241 0.634058 0.226293 -0.226293 -0.634058 -0.916241 -1.01695 0.916241 0.825504 0.571267 0.203883 -0.203883 -0.571267 -0.825504 -0.916241 0.634058 0.571267 0.395329 0.141091 -0.141091 -0.395329 -0.571267 -0.634058 0.226293 0.203883 0.141091 0.0503549 -0.0503549 -0.141091 -0.↩ 203883 -0.226293 -0.226293 -0.203883 -0.141091 -0.0503549 0.0503549 0.141091 0.203883 0.226293 -0.634058 -0.571267 -0.395329 -0.141091 0.141091 0.395329 0.571267 0.634058 -0.916241 -0.825504 -0.571267 -0.203883 0.203883 0.571267 0.825504 0.916241 -1.01695 -0.916241 -0.634058 -0.226293 0.226293 0.634058 0.916241 1.01695 Residue error: 9.55151e-07 Total iterations: 117 Exact solution u_exact: 1 0.900969 0.62349 0.222521

-0.222521 -0.62349 -0.900969 -1 0.900969 0.811745 0.561745 0.200484 -0.200484 -0.561745 -0.811745 -0.↩
900969 0.62349 0.561745 0.38874 0.13874 -0.13874 -0.38874 -0.561745 -0.62349 0.222521 0.200484 0.13874
0.0495156 -0.0495156 -0.13874 -0.200484 -0.222521 -0.222521 -0.200484 -0.13874 -0.0495156 0.0495156 0.↩
13874 0.200484 0.222521 -0.62349 -0.561745 -0.38874 -0.13874 0.13874 0.38874 0.561745 0.62349 -0.900969
-0.811745 -0.561745 -0.200484 0.200484 0.561745 0.811745 0.900969 -1 -0.900969 -0.62349 -0.222521 0.222521
0.62349 0.900969 1

=== Running Test 3: ROBIN SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0)
(0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.↩
142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857)
(0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286,
0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩
428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429)
(0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429)
(0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩
714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857,
0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩
857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1)
(1, 1) Discrete solution u_h: 1.00189 1.15522 1.3321 1.53615 1.77152 2.043 2.35609 2.71709 1.15522 1.33202
1.53598 1.77126 2.04267 2.35574 2.71681 3.13323 1.3321 1.53598 1.77119 2.04253 2.35556 2.71664 3.13315
3.61356 1.53615 1.77126 2.04253 2.3555 2.71656 3.13308 3.61356 4.16782 1.77152 2.04267 2.35556 2.71656
3.13306 3.61355 4.16785 4.80728 2.043 2.35574 2.71664 3.13308 3.61355 4.16785 4.80731 5.545 2.35609 2.↩
71681 3.13315 3.61356 4.16785 4.80731 5.54503 6.39606 2.71709 3.13323 3.61356 4.16782 4.80728 5.545 6.↩
39606 7.37784 Residue error: 9.88424e-07 Total iterations: 686 Exact solution u_exact: 1 1.15356 1.33071 1.53506
1.77079 2.04273 2.35642 2.71828 1.15356 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 1.33071
1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571
3.61725 4.17273 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 2.04273 2.35642 2.71828
3.13571 3.61725 4.17273 4.81352 5.55271 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541
2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541 7.38906

=== Running Test 4: DIRICHLET PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714,
0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714,
0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857)
(0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286,
0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩
428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429)
(0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429)
(0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩
714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857,
0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩
857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1,
1) Discrete solution u_h: 0 0 0 0 0 0 0 0 0 0.191446 0.344974 0.430175 0.430175 0.344974 0.191446 0 0 0.344974
0.621621 0.775148 0.775148 0.621621 0.344974 0 0 0.430175 0.775148 0.966594 0.966594 0.775148 0.430175
0 0 0.430175 0.775148 0.966594 0.966594 0.775148 0.430175 0 0 0.344974 0.621621 0.775148 0.775148 0.↩
621621 0.344974 0 0 0.191446 0.344974 0.430175 0.430175 0.344974 0.191446 0 0 0 0 0 0 0 0 0 Residue error:
9.15183e-07 Total iterations: 115 Exact solution u_exact: 0 0 0 0 0 0 0 0 0 0.188255 0.339224 0.423005 0.423005
0.339224 0.188255 5.31354e-17 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩
423005 0.762229 0.950484 0.950484 0.762229 0.423005 1.19394e-16 0 0.423005 0.762229 0.950484 0.950484
0.762229 0.423005 1.19394e-16 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩
188255 0.339224 0.423005 0.423005 0.339224 0.188255 5.31354e-17 0 5.31354e-17 9.57467e-17 1.19394e-16
1.19394e-16 9.57467e-17 5.31354e-17 1.49976e-32

=== Running Test 5: NEUMANN PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0)
(0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.↩
142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857)
(0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286,
0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩
428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429)
(0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429)

(0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1.01695 0.916241 0.634058 0.226293 -0.226293 -0.634058 -0.916241 -1.01695 0.916241 0.825504 0.571267 0.203883 -0.203883 -0.571267 -0.825504 -0.916241 0.634058 0.571267 0.395329 0.141091 -0.141091 -0.395329 -0.571267 -0.634058 0.226293 0.203883 0.141091 0.0503549 -0.0503549 -0.141091 -0.↩ 203883 -0.226293 -0.226293 -0.203883 -0.141091 -0.0503549 0.0503549 0.141091 0.203883 0.226293 -0.634058 -0.571267 -0.395329 -0.141091 0.141091 0.395329 0.571267 0.634058 -0.916241 -0.825504 -0.571267 -0.203883 0.203883 0.571267 0.825504 0.916241 -1.01695 -0.916241 -0.634058 -0.226293 0.226293 0.634058 0.916241 1.01695 Residue error: 9.55151e-07 Total iterations: 117 Exact solution u_exact: 1 0.900969 0.62349 0.222521 -0.222521 -0.62349 -0.900969 -1 0.900969 0.811745 0.561745 0.200484 -0.200484 -0.561745 -0.811745 -0.↩ 900969 0.62349 0.561745 0.38874 0.13874 -0.13874 -0.38874 -0.561745 -0.62349 0.222521 0.200484 0.13874 0.0495156 -0.0495156 -0.13874 -0.200484 -0.222521 -0.222521 -0.200484 -0.13874 -0.0495156 0.0495156 0.↩ 13874 0.200484 0.222521 -0.62349 -0.561745 -0.38874 -0.13874 0.13874 0.38874 0.561745 0.62349 -0.900969 -0.811745 -0.561745 -0.200484 0.200484 0.561745 0.811745 0.900969 -1 -0.900969 -0.62349 -0.222521 0.222521 0.62349 0.900969 1

=== Running Test 6: ROBIN PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.↩ 428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.↩ 142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1.00189 1.15522 1.3321 1.53615 1.77152 2.043 2.35609 2.71709 1.15522 1.33202 1.53598 1.77126 2.04267 2.35574 2.71681 3.13323 1.3321 1.53598 1.77119 2.04253 2.35556 2.71664 3.13315 3.61356 1.53615 1.77126 2.04253 2.3555 2.71656 3.13308 3.61356 4.16782 1.77152 2.04267 2.35556 2.71656 3.13306 3.61355 4.16785 4.80728 2.043 2.35574 2.71664 3.13308 3.61355 4.16785 4.80731 5.545 2.35609 2.↩ 71681 3.13315 3.61356 4.16785 4.80731 5.54503 6.39606 2.71709 3.13323 3.61356 4.16782 4.80728 5.545 6.↩ 39606 7.37784 Residue error: 9.91669e-07 Total iterations: 686 Exact solution u_exact: 1 1.15356 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 1.15356 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541 7.38906

=== Running Test 7: DIRICHLET PARALLEL (Schwarz) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 0 0 0 0 0 0 0 0 0 0.191446 0.344975 0.430176 0.430176 0.344975 0.191446 0 0 0.344975 0.621623 0.775151 0.775151 0.621623 0.344975 0 0 0.430176 0.775151 0.966597 0.966597 0.775151 0.430176 0 0 0.430176 0.775151 0.966597 0.966597 0.775151 0.430176 0 0 0.344975 0.621623 0.775151 0.775151 0.↩ 621623 0.344975 0 0 0.191446 0.344975 0.430176 0.430176 0.344975 0.191446 0 0 0 0 0 0 0 0 0 Residue error: 8.90472e-07 Total iterations: 64 Exact solution u_exact: 0 0 0 0 0 0 0 0 0 0.188255 0.339224 0.423005 0.423005 0.339224 0.188255 5.31354e-17 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩ 423005 0.762229 0.950484 0.950484 0.762229 0.423005 1.19394e-16 0 0.423005 0.762229 0.950484 0.950484

0.762229 0.423005 1.19394e-16 0 0.339224 0.61126 0.762229 0.762229 0.61126 0.339224 9.57467e-17 0 0.↩
188255 0.339224 0.423005 0.423005 0.339224 0.188255 5.31354e-17 0 5.31354e-17 9.57467e-17 1.19394e-16
1.19394e-16 9.57467e-17 5.31354e-17 1.49976e-32

=== Running Test 8: NEUMANN PARALLEL (Schwarz) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩
428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩
714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩
857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1.01695 0.916243 0.63406 0.226293 -0.226293 -0.63406 -0.916243 -1.01695 0.916243 0.825506 0.571268 0.203883 -0.203883 -0.571268 -0.825506 -0.916243 0.63406 0.571268 0.39533 0.141092 -0.141092 -0.39533 -0.571268 -0.63406 0.226293 0.203883 0.141092 0.050355 -0.050355 -0.141092 -0.203883 -0.226293 -0.226293 -0.203883 -0.141092 -0.050355 0.050355 0.141092 0.203883 0.226293 -0.63406 -0.571268 -0.39533 -0.141092 0.141092 0.39533 0.571268 0.63406 -0.916243 -0.825506 -0.571268 -0.203883 0.203883 0.↩
571268 0.825506 0.916243 -1.01695 -0.916243 -0.63406 -0.226293 0.226293 0.63406 0.916243 1.01695 Residue error: 9.38636e-07 Total iterations: 65 Exact solution u_exact: 1 0.900969 0.62349 0.222521 -0.222521 -0.62349 -0.900969 -1 0.900969 0.811745 0.561745 0.200484 -0.200484 -0.561745 -0.811745 -0.900969 0.62349 0.561745 0.38874 0.13874 -0.13874 -0.38874 -0.561745 -0.62349 0.222521 0.200484 0.13874 0.0495156 -0.0495156 -0.↩
13874 -0.200484 -0.222521 -0.222521 -0.200484 -0.13874 -0.0495156 0.0495156 0.13874 0.200484 0.222521 -0.62349 -0.561745 -0.38874 -0.13874 0.13874 0.38874 0.561745 0.62349 -0.900969 -0.811745 -0.561745 -0.↩
200484 0.200484 0.561745 0.811745 0.900969 -1 -0.900969 -0.62349 -0.222521 0.222521 0.62349 0.900969 1

=== Running Test 9: ROBIN PARALLEL (Schwarz) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.↩
142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩
428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩
714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩
857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1.0019 1.15523 1.33211 1.53616 1.77153 2.04301 2.3561 2.7171 1.15523 1.33203 1.53599 1.77127 2.04268 2.35575 2.71682 3.13324 1.33211 1.53599 1.7712 2.04254 2.35557 2.71665 3.13316 3.61357 1.53616 1.77127 2.04254 2.35551 2.71657 3.13309 3.61357 4.16783 1.77153 2.04269 2.35557 2.71657 3.13307 3.61356 4.16786 4.80729 2.04301 2.35575 2.71665 3.13309 3.61356 4.16786 4.80732 5.54501 2.3561 2.71682 3.13316 3.61357 4.16786 4.80732 5.54504 6.39607 2.7171 3.13324 3.61357 4.16783 4.80729 5.54501 6.39607 7.37785 Residue error: 9.78087e-07 Total iterations: 367 Exact solution u_exact: 1 1.15356 1.33071 1.↩
53506 1.77079 2.04273 2.35642 2.71828 1.15356 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 1.33071 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 1.53506 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 1.77079 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 2.04273 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 2.35642 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541 2.71828 3.13571 3.61725 4.17273 4.81352 5.55271 6.40541 7.38906

=== Running Test 10: DIRICHLET SEQUENTIAL vs PARALLEL (Jacobi) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 0.0224216 | 0.0224216 | 115 | 115 | 0.127903 | 0.705374 | 0.181327 16 | 0.00705013 | 0.00705013 | 484 | 484 | 2.81779 | 1.83588 | 1.53485 32 | 0.00219087 | 0.00219087 | 1862 | 1862 | 40.3516 | 10.3313 | 3.90574 64 | 1.92326e-05 | 1.92326e-05 | 6838 | 6838 | 609.376 | 113.708 | 5.35911 128 | 0.00297931 | 0.00297931 | 24357 | 24357 | 8905.9 | 2167.65 | 4.10856

=== Running Test 11: NEUMANN SEQUENTIAL vs PARALLEL (Jacobi) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8

| 0.0288298 | 0.0288298 | 117 | 117 | 0.263287 | 1.03984 | 0.2532 16 | 0.00799538 | 0.00799538 | 489 | 489 | 4.14442 | 2.8185 | 1.47043 32 | 0.00234451 | 0.00234451 | 1874 | 1874 | 46.3196 | 16.0704 | 2.88229 64 | 4.↩ 52179e-05 | 4.52179e-05 | 6863 | 6863 | 656.96 | 169.914 | 3.86642 128 | 0.00297485 | 0.00297485 | 24408 | 24408 | 9540.64 | 1625.12 | 5.87075

=== Running Test 12: ROBIN SEQUENTIAL vs PARALLEL (Jacobi) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 0.0114873 | 0.0114874 | 686 | 686 | 0.796436 | 2.43911 | 0.326528 16 | 0.00338568 | 0.00338323 | 2826 | 2830 | 9.94529 | 26.9137 | 0.369526 32 | 0.00192132 | 0.00191688 | 10813 | 10818 | 102.008 | 84.9675 | 1.20056 64 | 0.00486434 | 0.00485938 | 39654 | 39659 | 1777.96 | 498.237 | 3.5685 128 | 0.166917 | 0.166917 | 100000 | 100000 | 16875.3 | 3629.13 | 4.64997

=== Running Test 13: DIRICHLET SEQUENTIAL vs PARALLEL (Schwarz) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 0.0224216 | 0.0224259 | 115 | 64 | 0.150057 | 1.03851 | 0.144492 16 | 0.00705013 | 0.0070833 | 484 | 149 | 3.10881 | 2.47869 | 1.25422 32 | 0.00219087 | 0.00234987 | 1862 | 394 | 48.6538 | 20.1838 | 2.41054 64 | 1.92326e-05 | 0.000711865 | 6838 | 1175 | 559.697 | 165.39 | 3.38411 128 | 0.00297931 | 0.000102604 | 24357 | 3743 | 8926.76 | 1880.75 | 4.7464

=== Running Test 14: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 0.0288298 | 0.0288342 | 117 | 65 | 0.318529 | 10.5872 | 0.0300864 16 | 0.00799538 | 0.00802948 | 489 | 137 | 4.15105 | 2.89251 | 1.4351 32 | 0.00234451 | 0.00250702 | 1874 | 367 | 51.875 | 17.6518 | 2.93879 64 | 4.52179e-05 | 0.000744086 | 6863 | 1112 | 729.591 | 172.333 | 4.23361 128 | 0.00297485 | 8.27345e-05 | 24408 | 3621 | 9867.55 | 1967.09 | 5.01632

=== Running Test 15: ROBIN SEQUENTIAL vs PARALLEL (Schwarz) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 0.0114873 | 0.0114708 | 686 | 367 | 0.82971 | 4.69127 | 0.176863 16 | 0.00338568 | 0.00326322 | 2826 | 819 | 10.1786 | 12.5318 | 0.812218 32 | 0.00192132 | 0.00114779 | 10813 | 2202 | 127.038 | 74.3448 | 1.70877 64 | 0.00486434 | 0.000881327 | 39654 | 6642 | 1669.63 | 586.381 | 2.84736 128 | 0.166917 | 0.00229855 | 100000 | 21376 | 17029.7 | 6106.27 | 2.78889

=== Running Test 16: DIRICHLET SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1 1.14286 1.28571 1.42857 1.57143 1.71429 1.85714 2 1.14286 1.29154 1.↩ 44606 1.60641 1.77259 1.9446 2.12245 2.30612 1.28571 1.44606 1.61807 1.80174 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80174 2.01457 2.24489 2.49271 2.75801 3.04082 1.57143 1.77259 1.99708 2.24489 2.51603 2.81049 3.12828 3.46939 1.71429 1.9446 2.20408 2.49271 2.81049 3.15743 3.53353 3.93878 1.85714 2.12245 2.42274 2.75801 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5 Residue error: 9.25545e-07 Total iterations: 127 Exact solution u_exact: 1 1.14286 1.28571 1.42857 1.57143 1.↩ 71429 1.85714 2 1.14286 1.29155 1.44606 1.60641 1.77259 1.94461 2.12245 2.30612 1.28571 1.44606 1.61808 1.80175 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80175 2.01458 2.2449 2.49271 2.75802 3.04082 1.57143 1.77259 1.99708 2.2449 2.51603 2.8105 3.12828 3.46939 1.71429 1.94461 2.20408 2.49271 2.8105 3.↩ 15743 3.53353 3.93878 1.85714 2.12245 2.42274 2.75802 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5

=== Running Test 17: NEUMANN SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857)

(0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: -1.33694 -1.19367 -1.05123 -0.907955 -0.765514 -0.622241 -0.4798 -0.336526 -1.↩ 19367 -1.0454 -0.890462 -0.730529 -0.563932 -0.392337 -0.214077 -0.0308205 -1.05123 -0.890462 -0.718867 -0.534777 -0.339858 -0.132445 0.0857976 0.316535 -0.907955 -0.730529 -0.534777 -0.322366 -0.0916285 0.↩ 155768 0.421491 0.703873 -0.765514 -0.563932 -0.339858 -0.0916285 0.179092 0.473969 0.791337 1.13286 -0.622241 -0.392337 -0.132445 0.155768 0.473969 0.820491 1.197 1.60183 -0.4798 -0.214077 0.0857976 0.↩ 421491 0.791337 1.197 1.63682 2.11245 -0.336526 -0.0308205 0.316535 0.703873 1.13286 1.60183 2.11245 2.66306 Residue error: 0.0617213 Total iterations: 100000 Exact solution u_exact: 1 1.14286 1.28571 1.42857 1.57143 1.71429 1.85714 2 1.14286 1.29155 1.44606 1.60641 1.77259 1.94461 2.12245 2.30612 1.28571 1.↩ 44606 1.61808 1.80175 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80175 2.01458 2.2449 2.49271 2.75802 3.04082 1.57143 1.77259 1.99708 2.2449 2.51603 2.8105 3.12828 3.46939 1.71429 1.94461 2.20408 2.49271 2.8105 3.15743 3.53353 3.93878 1.85714 2.12245 2.42274 2.75802 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5

=== Running Test 18: ROBIN SEQUENTIAL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.↩ 142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: -0.00190863 0.42639 0.85179 1.27145 1.68264 2.08282 2.46967 2.84115 0.569248 0.994667 1.41433 1.82553 2.22569 2.61251 2.98393 3.33822 1.1375 1.55719 1.96839 2.36856 2.75536 3.12675 3.48098 3.81667 1.70002 2.11124 2.51141 2.89821 3.26959 3.62379 3.95944 4.27551 2.25407 2.65427 3.04107 3.41244 3.76664 4.10226 4.4183 4.71415 2.79711 3.18394 3.55532 3.90951 4.24512 4.56114 4.85696 5.13238 3.32681 3.69822 4.05241 4.38801 4.70402 4.99982 5.27522 5.53041 3.84115 4.19536 4.53095 4.84694 5.14272 5.41809 5.67327 5.90887 Residue error: 9.87596e-07 Total iterations: 693 Exact solution u_exact: 0 0.428086 0.853271 1.27271 1.68369 2.08365 2.47026 2.84147 0.570943 0.996129 1.41557 1.82655 2.22651 2.61312 2.↩ 98433 3.33839 1.13899 1.55843 1.96941 2.36936 2.75598 3.12719 3.48125 3.81678 1.70129 2.11226 2.51222 2.89883 3.27004 3.62411 3.95964 4.27562 2.25512 2.65508 3.04169 3.4129 3.76697 4.1025 4.41847 4.71429 2.79794 3.18455 3.55576 3.90982 4.24535 4.56133 4.85714 5.13258 3.3274 3.69861 4.05268 4.38821 4.70419 5 5.27544 5.53071 3.84147 4.19554 4.53107 4.84705 5.14286 5.41829 5.67357 5.9093

=== Running Test 19: DIRICHLET PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.↩ 428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.↩ 714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.↩ 857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: 1 1.14286 1.28571 1.42857 1.57143 1.71429 1.85714 2 1.14286 1.29154 1.↩ 44606 1.60641 1.77259 1.9446 2.12245 2.30612 1.28571 1.44606 1.61807 1.80174 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80174 2.01457 2.24489 2.49271 2.75801 3.04082 1.57143 1.77259 1.99708 2.24489 2.51603 2.81049 3.12828 3.46939 1.71429 1.9446 2.20408 2.49271 2.81049 3.15743 3.53353 3.93878 1.85714

2.12245 2.42274 2.75801 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5 Residue error: 9.25545e-07 Total iterations: 127 Exact solution u_exact: 1 1.14286 1.28571 1.42857 1.57143 1.71429 1.85714 2 1.14286 1.29155 1.44606 1.60641 1.77259 1.94461 2.12245 2.30612 1.28571 1.44606 1.61808 1.80175 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80175 2.01458 2.2449 2.49271 2.75802 3.04082 1.57143 1.77259 1.99708 2.2449 2.51603 2.8105 3.12828 3.46939 1.71429 1.94461 2.20408 2.49271 2.8105 3.15743 3.53353 3.93878 1.85714 2.12245 2.42274 2.75802 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5

=== Running Test 20: NEUMANN PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: -1.35735 -1.21408 -1.07164 -0.928363 -0.785923 -0.642649 -0.500208 -0.356935 -1.21408 -1.06581 -0.91087 -0.750937 -0.58434 -0.412745 -0.234486 -0.0512287 -1.07164 -0.91087 -0.739275 -0.555185 -0.360267 -0.152853 0.0653894 0.296127 -0.928363 -0.750937 -0.555185 -0.342774 -0.112037 0.13536 0.401083 0.683465 -0.785923 -0.58434 -0.360267 -0.112037 0.158684 0.453561 0.770929 1.11245 -0.642649 -0.412745 -0.152853 0.13536 0.453561 0.800083 1.17659 1.58142 -0.500208 -0.234486 0.0653894 0.401083 0.770929 1.17659 1.61641 2.09204 -0.356935 -0.0512287 0.296127 0.683465 1.11245 1.58142 2.09204 2.64265 Residue error: 0.00125936 Total iterations: 100000 Exact solution u_exact: 1 1.14286 1.28571 1.42857 1.57143 1.71429 1.85714 2 1.14286 1.29155 1.44606 1.60641 1.77259 1.94461 2.12245 2.30612 1.28571 1.44606 1.61808 1.80175 1.99708 2.20408 2.42274 2.65306 1.42857 1.60641 1.80175 2.01458 2.2449 2.49271 2.75802 3.04082 1.57143 1.77259 1.99708 2.2449 2.51603 2.8105 3.12828 3.46939 1.71429 1.94461 2.20408 2.49271 2.8105 3.15743 3.53353 3.93878 1.85714 2.12245 2.42274 2.75802 3.12828 3.53353 3.97376 4.44898 2 2.30612 2.65306 3.04082 3.46939 3.93878 4.44898 5

=== Running Test 21: ROBIN PARALLEL (Jacobi) === Mesh points (x, y): (0, 0) (0.142857, 0) (0.285714, 0) (0.428571, 0) (0.571429, 0) (0.714286, 0) (0.857143, 0) (1, 0) (0, 0.142857) (0.142857, 0.142857) (0.285714, 0.142857) (0.428571, 0.142857) (0.571429, 0.142857) (0.714286, 0.142857) (0.857143, 0.142857) (1, 0.142857) (0, 0.285714) (0.142857, 0.285714) (0.285714, 0.285714) (0.428571, 0.285714) (0.571429, 0.285714) (0.714286, 0.285714) (0.857143, 0.285714) (1, 0.285714) (0, 0.428571) (0.142857, 0.428571) (0.285714, 0.428571) (0.428571, 0.428571) (0.571429, 0.428571) (0.714286, 0.428571) (0.857143, 0.428571) (1, 0.428571) (0, 0.571429) (0.142857, 0.571429) (0.285714, 0.571429) (0.428571, 0.571429) (0.571429, 0.571429) (0.714286, 0.571429) (0.857143, 0.571429) (1, 0.571429) (0, 0.714286) (0.142857, 0.714286) (0.285714, 0.714286) (0.428571, 0.714286) (0.571429, 0.714286) (0.714286, 0.714286) (0.857143, 0.714286) (1, 0.714286) (0, 0.857143) (0.142857, 0.857143) (0.285714, 0.857143) (0.428571, 0.857143) (0.571429, 0.857143) (0.714286, 0.857143) (0.857143, 0.857143) (1, 0.857143) (0, 1) (0.142857, 1) (0.285714, 1) (0.428571, 1) (0.571429, 1) (0.714286, 1) (0.857143, 1) (1, 1) Discrete solution u_h: -0.00190866 0.42639 0.851789 1.27145 1.68264 2.08282 2.46967 2.84115 0.569248 0.994666 1.41433 1.82553 2.22569 2.61251 2.98393 3.33822 1.1375 1.55719 1.96839 2.36856 2.75536 3.12675 3.48098 3.81667 1.70002 2.11124 2.51141 2.89821 3.26959 3.62379 3.95944 4.27551 2.25407 2.65427 3.04107 3.41244 3.76664 4.10226 4.4183 4.71415 2.79711 3.18394 3.55532 3.90951 4.24512 4.56114 4.85696 5.13238 3.32681 3.69822 4.05241 4.38801 4.70402 4.99982 5.27522 5.53041 3.84115 4.19536 4.53095 4.84694 5.14272 5.41809 5.67327 5.90887 Residue error: 9.87178e-07 Total iterations: 693 Exact solution u_exact: 0 0.428086 0.853271 1.27271 1.68369 2.08365 2.47026 2.84147 0.570943 0.996129 1.41557 1.82655 2.22651 2.61312 2.98433 3.33839 1.13899 1.55843 1.96941 2.36936 2.75598 3.12719 3.48125 3.81678 1.70129 2.11226 2.51222 2.89883 3.27004 3.62411 3.95964 4.27562 2.25512 2.65508 3.04169 3.4129 3.76697 4.1025 4.41847 4.71429 2.79794 3.18455 3.55576 3.90982 4.24535 4.56133 4.85714 5.13258 3.3274 3.69861 4.05268 4.38821 4.70419 5 5.27544 5.53071 3.84147 4.19554 4.53107 4.84705 5.14286 5.41829 5.67357 5.9093

=== Running Test 22: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz) === Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup 8 | 7.06562 | 7.12733 | 100000 | 5954 | 44.9544 | 86.0416 | 0.522472 16 | 9.64249 | 9.68533 | 100000 | 273 | 170.686 | 4.23925 | 40.2632 32 | 13.4115 | 13.4414 | 100000 | 726 | 627.354 | 22.4681 | 27.922 64 | 18.8146

| 18.8356 | 100000 | 2192 | 2303.21 | 122.13 | 18.8588 128 | 26.5025 | 26.5173 | 100000 | 7075 | 8777.83 | 1205.8 | 7.27964

Chapter 3

Namespace Index

3.1 Namespace List

Here is a list of all namespaces with brief descriptions:

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

Chapter 6

Namespace Documentation

6.1 laplacian_solvers Namespace Reference

Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

Classes

- struct Convergence_Struct

  Stores collected benchmark metrics across multiple grid refinement levels.
- class Convergence_Test

  Benchmark coordinator running serial and parallel execution comparison groups.
- class Laplacian_Solver

  Core class for solving the Laplace/Poisson equation on structured grids.

Typedefs

- using funcType = std::function< double(double, double)>

  Type alias for standard 2D spatial functions.

Functions

- Data_Struct< funcType > create_default_data ()

  Generates a baseline configuration structure with default parameters.
- void run_test_1_dirichlet_sequential (int mpi_rank)

  Test 1: Evaluates a sequential Jacobi solver under homogeneous Dirichlet boundaries.
- void run_test_2_neumann_sequential (int mpi_rank)

  Test 2: Evaluates a sequential Jacobi solver under pure Neumann boundaries.
- void run_test_3_robin_sequential (int mpi_rank)

  Test 3: Evaluates a sequential Jacobi solver under Robin boundary conditions.
- void run_test_4_dirichlet_parallel (int mpi_rank)

  Test 4: Evaluates a parallel MPI Jacobi solver under Dirichlet boundaries.
- void run_test_5_neumann_parallel (int mpi_rank)

  Test 5: Evaluates a parallel MPI Jacobi solver under Neumann boundaries.
- void run_test_6_robin_parallel (int mpi_rank)

Test 6: Evaluates a parallel MPI Jacobi solver under Robin boundaries.

- void run_test_7_dirichlet_parallel_schwarz (int mpi_rank)

  Test 7: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Dirichlet boundaries.
- void run_test_8_neumann_parallel_schwarz (int mpi_rank)

  Test 8: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Neumann boundaries.
- void run_test_9_robin_parallel_schwarz (int mpi_rank)

  Test 9: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Robin boundaries.
- void run_test_10_dirichlet_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 10: Performs error convergence analysis and speedup profiling for Dirichlet Jacobi routines.
- void run_test_11_neumann_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 11: Performs error convergence analysis and speedup profiling for Neumann Jacobi routines.
- void run_test_12_robin_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 12: Performs error convergence analysis and speedup profiling for Robin Jacobi routines.
- void run_test_13_dirichlet_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 13: Performs error convergence analysis and speedup profiling for Dirichlet Schwarz domain decomposition.
- void run_test_14_neumann_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 14: Performs error convergence analysis and speedup profiling for Neumann Schwarz domain decomposition.
- void run_test_15_robin_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 15: Performs error convergence analysis and speedup profiling for Robin Schwarz domain decomposition.
- void run_test_16_dirichlet_sequential (int mpi_rank)

  Test 16: Sequential Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.
- void run_test_17_neumann_sequential (int mpi_rank)

  Test 17: Sequential Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.
- void run_test_18_robin_sequential (int mpi_rank)

  Test 18: Sequential Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.
- void run_test_19_dirichlet_parallel (int mpi_rank)

  Test 19: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.
- void run_test_20_neumann_parallel (int mpi_rank)

  Test 20: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.
- void run_test_21_robin_parallel (int mpi_rank)

  Test 21: Parallel MPI-distributed Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.
- void run_test_22_neumann_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 22: Performance convergence benchmark for a polynomial solution under Neumann conditions using the Schwarz solver.

6.1.1 Detailed Description

Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

6.1.2 Typedef Documentation

6.1.2.1 funcType

```
using laplacian_solvers::funcType = typedef std::function<double(double, double)>
```

Type alias for standard 2D spatial functions.

6.1.3 Function Documentation

6.1.3.1 create_default_data()

```
Data_Struct< funcType > laplacian_solvers::create_default_data ( )  [inline]
```

Generates a baseline configuration structure with default parameters.

Returns

Initialized Data_Struct with standard numerical tolerances and grid boundaries.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.2 run_test_10_dirichlet_sequential_vs_parallel_jacobi()

```
void laplacian_solvers::run_test_10_dirichlet_sequential_vs_parallel_jacobi (
            int mpi_rank )  [inline]
```

Test 10: Performs error convergence analysis and speedup profiling for Dirichlet Jacobi routines.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.3 run_test_11_neumann_sequential_vs_parallel_jacobi()

```
void laplacian_solvers::run_test_11_neumann_sequential_vs_parallel_jacobi (
            int mpi_rank )  [inline]
```

Test 11: Performs error convergence analysis and speedup profiling for Neumann Jacobi routines.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.4 run_test_12_robin_sequential_vs_parallel_jacobi()

```
void laplacian_solvers::run_test_12_robin_sequential_vs_parallel_jacobi (
            int mpi_rank )  [inline]
```

Test 12: Performs error convergence analysis and speedup profiling for Robin Jacobi routines.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.5 run_test_13_dirichlet_sequential_vs_parallel_schwarz()

```
void laplacian_solvers::run_test_13_dirichlet_sequential_vs_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 13: Performs error convergence analysis and speedup profiling for Dirichlet Schwarz domain decomposition.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.6 run_test_14_neumann_sequential_vs_parallel_schwarz()

```
void laplacian_solvers::run_test_14_neumann_sequential_vs_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 14: Performs error convergence analysis and speedup profiling for Neumann Schwarz domain decomposition.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.7 run_test_15_robin_sequential_vs_parallel_schwarz()

```
void laplacian_solvers::run_test_15_robin_sequential_vs_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 15: Performs error convergence analysis and speedup profiling for Robin Schwarz domain decomposition.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.8 run_test_16_dirichlet_sequential()

```
void laplacian_solvers::run_test_16_dirichlet_sequential (
            int mpi_rank )  [inline]
```

Test 16: Sequential Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.

- Validates analytical solution: $u(x, y) = x^2y + xy^2 + x + y + 1$

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.9 run_test_17_neumann_sequential()

```
void laplacian_solvers::run_test_17_neumann_sequential (
            int mpi_rank )  [inline]
```

Test 17: Sequential Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.

- Verifies normal derivative flux mappings: $\frac{\partial u}{\partial n} = \nabla u \cdot \mathbf{n}$

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.10 run_test_18_robin_sequential()

```
void laplacian_solvers::run_test_18_robin_sequential (
            int mpi_rank )  [inline]
```

Test 18: Sequential Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.

- Evaluates linear combination constraints: $u + \frac{\partial u}{\partial n} = g$

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.11 run_test_19_dirichlet_parallel()

```
void laplacian_solvers::run_test_19_dirichlet_parallel (
            int mpi_rank ) [inline]
```

Test 19: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.12 run_test_1_dirichlet_sequential()

```
void laplacian_solvers::run_test_1_dirichlet_sequential (
            int mpi_rank ) [inline]
```

Test 1: Evaluates a sequential Jacobi solver under homogeneous Dirichlet boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.13 run_test_20_neumann_parallel()

```
void laplacian_solvers::run_test_20_neumann_parallel (
            int mpi_rank ) [inline]
```

Test 20: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.14 run_test_21_robin_parallel()

```
void laplacian_solvers::run_test_21_robin_parallel (
            int mpi_rank ) [inline]
```

Test 21: Parallel MPI-distributed Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.15 run_test_22_neumann_sequential_vs_parallel_schwarz()

```
void laplacian_solvers::run_test_22_neumann_sequential_vs_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 22: Performance convergence benchmark for a polynomial solution under Neumann conditions using the Schwarz solver.

Parameters

mpi_rank	Rank of the calling MPI process.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.16 run_test_2_neumann_sequential()

```
void laplacian_solvers::run_test_2_neumann_sequential (
            int mpi_rank )  [inline]
```

Test 2: Evaluates a sequential Jacobi solver under pure Neumann boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.17 run_test_3_robin_sequential()

```
void laplacian_solvers::run_test_3_robin_sequential (
            int mpi_rank )  [inline]
```

Test 3: Evaluates a sequential Jacobi solver under Robin boundary conditions.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.18 run_test_4_dirichlet_parallel()

```
void laplacian_solvers::run_test_4_dirichlet_parallel (
            int mpi_rank ) [inline]
```

Test 4: Evaluates a parallel MPI Jacobi solver under Dirichlet boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.19 run_test_5_neumann_parallel()

```
void laplacian_solvers::run_test_5_neumann_parallel (
            int mpi_rank ) [inline]
```

Test 5: Evaluates a parallel MPI Jacobi solver under Neumann boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.20 run_test_6_robin_parallel()

```
void laplacian_solvers::run_test_6_robin_parallel (
            int mpi_rank ) [inline]
```

Test 6: Evaluates a parallel MPI Jacobi solver under Robin boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.21 run_test_7_dirichlet_parallel_schwarz()

```
void laplacian_solvers::run_test_7_dirichlet_parallel_schwarz (
            int mpi_rank ) [inline]
```

Test 7: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Dirichlet boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.22 run_test_8_neumann_parallel_schwarz()

```
void laplacian_solvers::run_test_8_neumann_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 8: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Neumann boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

6.1.3.23 run_test_9_robin_parallel_schwarz()

```
void laplacian_solvers::run_test_9_robin_parallel_schwarz (
            int mpi_rank )  [inline]
```

Test 9: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Robin boundaries.

Parameters

mpi_rank	Active process rank.

Here is the call graph for this function: Here is the caller graph for this function:

Chapter 7

Class Documentation

7.1 laplacian_solvers::Convergence_Struct Struct Reference

Stores collected benchmark metrics across multiple grid refinement levels.

`#include <laplacian_convergence_test.hpp>`

Public Attributes

- std::vector< double > errors_serial

  L2 norm error tracking for sequential solver runs.
- std::vector< double > errors_parallel

  L2 norm error tracking for parallel solver runs.
- std::vector< unsigned > n

  Grid resolution size sequence values.
- std::vector< double > iterations_serial

  Number of inner solver loops completed sequentially.
- std::vector< double > iterations_parallel

  Number of inner solver loops completed in parallel.
- std::vector< double > times_serial

  Elapsed execution duration for sequential loops (ms).
- std::vector< double > times_parallel

  Elapsed execution duration for parallel loops (ms).
- std::vector< double > speedups

  Speedup scaling metric ratio (Serial Time / Parallel Time).

7.1.1 Detailed Description

Stores collected benchmark metrics across multiple grid refinement levels.

- Tracks L2 errors, iteration counts, and computational times for both serial and parallel runs to facilitate convergence order verification and speedup analysis.

7.1.2 Member Data Documentation

7.1.2.1 errors_parallel

```
std::vector<double> laplacian_solvers::Convergence_Struct::errors_parallel
```

L2 norm error tracking for parallel solver runs.

7.1.2.2 errors_serial

```
std::vector<double> laplacian_solvers::Convergence_Struct::errors_serial
```

L2 norm error tracking for sequential solver runs.

7.1.2.3 iterations_parallel

```
std::vector<double> laplacian_solvers::Convergence_Struct::iterations_parallel
```

Number of inner solver loops completed in parallel.

7.1.2.4 iterations_serial

```
std::vector<double> laplacian_solvers::Convergence_Struct::iterations_serial
```

Number of inner solver loops completed sequentially.

7.1.2.5 n

```
std::vector<unsigned> laplacian_solvers::Convergence_Struct::n
```

Grid resolution size sequence values.

7.1.2.6 speedups

```
std::vector<double> laplacian_solvers::Convergence_Struct::speedups
```

Speedup scaling metric ratio (Serial Time / Parallel Time).

7.1.2.7 times_parallel

```
std::vector<double> laplacian_solvers::Convergence_Struct::times_parallel
```

Elapsed execution duration for parallel loops (ms).

7.1.2.8 times_serial

```
std::vector<double> laplacian_solvers::Convergence_Struct::times_serial
```

Elapsed execution duration for sequential loops (ms).

The documentation for this struct was generated from the following file:

- include/laplacian_convergence_test.hpp

7.2 laplacian_solvers::Convergence_Test< solver_type, boundary_condition, funcType > Class Template Reference

Benchmark coordinator running serial and parallel execution comparison groups.

```
#include <laplacian_convergence_test.hpp>
```

Collaboration diagram for laplacian_solvers::Convergence_Test< solver_type, boundary_condition, funcType >:

7.3 Data_Struct< funcType > Struct Template Reference

Holds all geometric, physical, and numerical configuration parameters for the problem.

```
#include <laplacian_solvers.hpp>
```

Public Attributes

- unsigned n
  Number of discretization subintervals per spatial coordinate.
- double x1
- double x2
- funcType f0
  Source term.
- funcType f1
  BC bottom edge.
- funcType f2
  BC right edge.
- funcType f3
  BC top edge.
- funcType f4
  BC left edge.
- funcType u_exact_lambda
  Analytical solution (for error analysis)
- double tolerance
  Convergence tolerance for the iterative solver.
- unsigned max_iterations
  Maximum number of iterations for the iterative solver.
- double alpha
  Parameter for Robin boundary condition: gamma∗u + du/dn = g.
- unsigned max_schwarz_iterations = 10
  Maximum number of Schwarz iterations (if using Schwarz method)

7.3.1 Detailed Description

template<typename funcType>
struct Data_Struct< funcType >

Holds all geometric, physical, and numerical configuration parameters for the problem.

Template Parameters

funcType	Type of the callables representing source terms and BCs.

7.3.2 Member Data Documentation

7.3.2.1 alpha

```
template<typename funcType >
double Data_Struct< funcType >::alpha
```

Parameter for Robin boundary condition: gamma∗u + du/dn = g.

7.3.2.2 f0

```
template<typename funcType >
funcType Data_Struct< funcType >::f0
```

Source term.

7.3.2.3 f1

```
template<typename funcType >
funcType Data_Struct< funcType >::f1
```

BC bottom edge.

7.3.2.4 f2

```
template<typename funcType >
funcType Data_Struct< funcType >::f2
```

BC right edge.

7.3.2.5 f3

```
template<typename funcType >
funcType Data_Struct< funcType >::f3
```

BC top edge.

7.3.2.6 f4

```
template<typename funcType >
funcType Data_Struct< funcType >::f4
```

BC left edge.

7.3.2.7 max_iterations

```
template<typename funcType >
unsigned Data_Struct< funcType >::max_iterations
```

Maximum number of iterations for the iterative solver.

7.3.2.8 max_schwarz_iterations

```
template<typename funcType >
unsigned Data_Struct< funcType >::max_schwarz_iterations = 10
```

Maximum number of Schwarz iterations (if using Schwarz method)

7.3.2.9 n

```
template<typename funcType >
unsigned Data_Struct< funcType >::n
```

Number of discretization subintervals per spatial coordinate.

7.3.2.10 tolerance

```
template<typename funcType >
double Data_Struct< funcType >::tolerance
```

Convergence tolerance for the iterative solver.

7.3.2.11 u_exact_lambda

```
template<typename funcType >
funcType Data_Struct< funcType >::u_exact_lambda
```

Analytical solution (for error analysis)

7.3.2.12 x1

```
template<typename funcType >
double Data_Struct< funcType >::x1
```

7.3.2.13 x2

```
template<typename funcType >
double Data_Struct< funcType >::x2
```

The documentation for this struct was generated from the following file:

- include/laplacian_solvers.hpp

7.4 laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType > Class Template Reference

Core class for solving the Laplace/Poisson equation on structured grids.

```
#include <laplacian_solvers.hpp>
```

Collaboration diagram for laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >:

Public Member Functions

- Laplacian_Solver (const Data_Struct< funcType > &d)
  Constructs a new Laplacian Solver object.
- Result_Struct solve ()
  Method that triggers the appropriate linear solver backend.
- void print () const
  Wrapper method to print the mesh, numerical solution, and exact solution.
- void export_to_vtk (const std::string &filename) const
  Exports global solver results to a structured ASCII VTK file format legacy format.
- void build_mesh ()
  Computes the uniform grid spacing and routes mesh generation to the active backend.

Private Member Functions

- void build_mesh_sequential ()
  Generates the global coordinates mesh on a single thread.
- void build_mesh_parallel ()
  Generates a local sub-grid slice allocated to the invoking MPI process.
- void process_filter ()
  Filters active MPI processes and checks communicator sizing validity.
- void build_exact_solution ()
  Allocates space for the exact analytical solution matrix and triggers initialization.
- void build_exact_solution_sequential ()
  Evaluates the exact analytical lambda expression sequentially across the full grid.
- void build_exact_solution_parallel ()
  Evaluates the exact analytical lambda function in parallel over the local grid slice.
- void print_mesh () const

Gathers and prints the grid mesh coordinates across all active nodes.

- void print_solution () const

  Prints the final discrete numerical solution array along with execution metadata.
- void print_exact_solution () const

  Gathers and prints the exact analytical benchmark evaluation solution array.
- Result_Struct parallel_solve ()

  Dispatches the parallel execution loop to the selected numerical algorithm.
- Result_Struct jacobi_sequential ()

  Solves the Laplacian problem sequentially using the Jacobi iterative method.
- Result_Struct jacobi_parallel ()

  Solves the Laplacian problem using a parallel hybrid Jacobi method.
- Result_Struct schwarz_parallel ()

  Solves the Laplacian problem using a parallel alternating Schwarz method.

Private Attributes

- Data_Struct< funcType > data

  Configuration of the problem and parameters.
- eigenMatrix meshX

  Matrix containing the X coordinates of the structured grid.
- eigenMatrix meshY

  Matrix containing the Y coordinates of the structured grid.
- double h

  Uniform grid spacing parameter h.
- eigenMatrix u_exact

  Analytical solution over the mesh points matrix.
- Result_Struct result

  Struct to hold the final results of the solver.
- int mpi_rank

  Rank of the MPI process (used in parallel execution).
- int mpi_size

  Number of participating MPI processes (used in parallel execution).
- int mpi_used_size

  Actual MPI_COMM_WORLD size (for reference).
- bool participating = true

  True if this rank participates in computation (rank < mpi_size)

7.4.1 Detailed Description

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType>
class laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >

Core class for solving the Laplace/Poisson equation on structured grids.

- Leverages compile-time metaprogramming (via template parameters) to statically configure the algorithm, boundary conditions, and execution backend, maximizing runtime efficiency.

Template Parameters

solver_type	The iterative algorithm to be used (SolverType).
boundary_condition	The type of BC applied to the domain (BoundaryCondition).
execution_mode	The computational paradigm backend (ExecutionMode).
funcType	The type of functional descriptors for sources and boundaries.

-

7.4.2 Constructor & Destructor Documentation

7.4.2.1 Laplacian_Solver()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType
>::Laplacian_Solver (
            const Data_Struct< funcType > & d )
```

Constructs a new Laplacian Solver object.

- Initializes problem data parameters, checks process topology validity via the filter, and allocates/populates both the computational grid grid and the exact solution matrix.

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Parallelism execution backend policy.
funcType	Callable type for boundary and source terms.

-

Parameters

d	Configuration data structure containing physical and analytical bounds.

-

Here is the call graph for this function:

7.4.3 Member Function Documentation

7.4.3.1 build_exact_solution()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
```

void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::build_exact_solution ( ) [private]

Allocates space for the exact analytical solution matrix and triggers initialization.

Here is the call graph for this function: Here is the caller graph for this function:

7.4.3.2 build_exact_solution_parallel()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::build_exact_solution_parallel ( ) [private]
```

Evaluates the exact analytical lambda function in parallel over the local grid slice.

- Re-allocates the local `u_exact` array to match the chunk sizing generated in build_mesh_parallel. Computations are offloaded to an active OpenMP thread pool operating on local memory rows.

Here is the call graph for this function:

7.4.3.3 build_exact_solution_sequential()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::build_exact_solution_sequential ( ) [private]
```

Evaluates the exact analytical lambda expression sequentially across the full grid.

Here is the call graph for this function:

7.4.3.4 build_mesh()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::build_mesh ( )
```

Computes the uniform grid spacing and routes mesh generation to the active backend.

- Calculates the uniform grid step $h = \frac{|x_2 - x_1|}{N-1}$. It guarantees correct computation even if domain bounds are specified in reversed spatial orientation ($x_2 < x_1$).

Here is the call graph for this function: Here is the caller graph for this function:

7.4.3.5 build_mesh_parallel()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::build_mesh_parallel ( )  [private]
```

Generates a local sub-grid slice allocated to the invoking MPI process.

- Distributes grid rows dynamically across active MPI processes. If the row count N is not perfectly divisible by the number of processes, the remainder rows are assigned one-by-one to the first lower-ranked processes to ensure optimal workload distribution. Internal cell row populations are accelerated locally via multi-threaded OpenMP parallel loops.

Here is the call graph for this function:

7.4.3.6 build_mesh_sequential()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::build_mesh_sequential ( )  [private]
```

Generates the global coordinates mesh on a single thread.

- Allocates and populates an $N \times N$ matrix representing the full grid domain layout, assigning spatial continuous coordinate mappings directly to `meshX` and `meshY`.

Here is the call graph for this function:

7.4.3.7 export_to_vtk()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::export_to_vtk (
            const std::string & filename ) const
```

Exports global solver results to a structured ASCII VTK file format legacy format.

- Gathers mesh positions and scalar field solutions from all active compute allocations via point-to-point gathering routines. Enforces directory tree setups and implements incremental filename counter adjustments to avoid unwanted file overrides. Output operations are pinned to Rank 0.

Parameters

filename	Target base name designation string for the exported VTK structure.

•

Here is the call graph for this function:

7.4.3.8 jacobi_parallel()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::jacobi_parallel ( )  [private]
```

Solves the Laplacian problem using a parallel hybrid Jacobi method.

- Splits the domain among MPI processes along rows. Each process allocates ghost/halo rows for neighbor data exchange. Within each process, calculations are threaded using OpenMP, and ghost layers are synchronized via non-blocking or point-to-point MPI primitives.

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Parallel execution policy backend.
funcType	Callable type for boundary and source terms.

•

Returns

• A Result_Struct containing the gathered solution on Rank 0, or partial local structs on other ranks.

Here is the call graph for this function:

7.4.3.9 jacobi_sequential()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::jacobi_sequential ( )  [private]
```

Solves the Laplacian problem sequentially using the Jacobi iterative method.

- Initializes the solution matrix, applies the initial boundary conditions, and runs the main relaxation loop until the squared L^2 norm error drops below the tolerance or the maximum iteration count is reached.

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Sequential execution policy backend.
funcType	Callable type for boundary and source terms.

- **Returns**
- A Result_Struct containing the final solution matrix, coordinates, and iteration metadata.

Here is the call graph for this function:

7.4.3.10 parallel_solve()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::parallel_solve ( )  [private]
```

Dispatches the parallel execution loop to the selected numerical algorithm.

- Performs compile-time static routing between a standard point-to-point Jacobi method or an overlapping/non-overlapping alternating Schwarz domain decomposition method.

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Parallelism execution backend policy.
funcType	Callable type for boundary and source terms.

- **Returns**
- A Result_Struct containing local or gathered compute states.

Here is the call graph for this function:

7.4.3.11 print()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::print ( ) const
```

Wrapper method to print the mesh, numerical solution, and exact solution.

7.4.3.12 print_exact_solution()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::print_exact_solution ( ) const  [private]
```

Gathers and prints the exact analytical benchmark evaluation solution array.

Here is the call graph for this function:

7.4.3.13 print_mesh()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::print_mesh ( ) const  [private]
```

Gathers and prints the grid mesh coordinates across all active nodes.

- If operating in parallel execution mode, a collective `MPI_Gatherv` operation collects the partitioned matrix layout components back into Rank 0 before rendering console output.

Here is the call graph for this function:

7.4.3.14 print_solution()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::print_solution ( ) const  [private]
```

Prints the final discrete numerical solution array along with execution metadata.

- Outputs metrics such as residual errors and convergence loop iteration counts. Execution output is restricted solely to the root rank (Rank 0).

Here is the call graph for this function:

7.4.3.15 process_filter()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
void laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::process_filter ( )  [inline], [private]
```

Filters active MPI processes and checks communicator sizing validity.

- In parallel execution mode, ensures that the number of allocated MPI ranks does not exceed the total number of mesh rows (N), preventing empty or unassigned processes. If the topology is invalid, Rank 0 prints an error and aborts execution.

Here is the call graph for this function: Here is the caller graph for this function:

7.4.3.16 schwarz_parallel()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::schwarz_parallel ( )  [private]
```

Solves the Laplacian problem using a parallel alternating Schwarz method.

- Divides the domain into subdomains allocated to different MPI ranks. The algorithm alternates between global macro-steps (inter-process halo updates via MPI) and local micro-steps (multi-threaded relaxation loops via OpenMP confined to the subdomains).

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Parallel execution policy backend.
funcType	Callable type for boundary and source terms.

- **Returns**
- A Result_Struct containing the assembled solution on Rank 0, or partial fields on other ranks.

Here is the call graph for this function:

7.4.3.17 solve()

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::solve ( )
```

Method that triggers the appropriate linear solver backend.

- Checks process validity and uses compile-time template routing (`if constexpr`) to dispatch execution based on the configured ExecutionMode.

Template Parameters

solver_type	Iterative algorithm selector.
boundary_condition	Boundary condition policy.
execution_mode	Parallelism execution backend policy.
funcType	Callable type for boundary and source terms.

- **Returns**
- A Result_Struct containing the final solution state.

Note

- Non-participating MPI processes or non-zero ranks under sequential mode will immediately return an uninitialized or incomplete result object.

Here is the call graph for this function: Here is the caller graph for this function:

7.4.4 Member Data Documentation

7.4.4.1 data

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
```

Data_Struct<funcType> laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::data [private]

Configuration of the problem and parameters.

7.4.4.2 h

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
double laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::h [private]

Uniform grid spacing parameter h.

7.4.4.3 meshX

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
eigenMatrix laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::meshX [private]

Matrix containing the X coordinates of the structured grid.

7.4.4.4 meshY

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
eigenMatrix laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::meshY [private]

Matrix containing the Y coordinates of the structured grid.

7.4.4.5 mpi_rank

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
int laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::mpi_rank [private]

Rank of the MPI process (used in parallel execution).

7.4.4.6 mpi_size

template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
int laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >::mpi_size [private]

Number of participating MPI processes (used in parallel execution).

7.4.4.7 mpi_used_size

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
int laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::mpi_used_size  [private]
```

Actual MPI_COMM_WORLD size (for reference).

7.4.4.8 participating

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
bool laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::participating = true  [private]
```

True if this rank participates in computation (rank < mpi_size)

7.4.4.9 result

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
Result_Struct laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::result  [private]
```

Struct to hold the final results of the solver.

7.4.4.10 u_exact

```
template<SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode execution↩
_mode, typename funcType >
eigenMatrix laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode,
funcType >::u_exact  [private]
```

Analytical solution over the mesh points matrix.

The documentation for this class was generated from the following files:

- include/laplacian_solvers.hpp
- include/laplacian_solvers_implementation.hpp
- include/laplacian_solvers_initializer.hpp
- include/laplacian_solvers_parallel_jacobi_implementation.hpp
- include/laplacian_solvers_parallel_schwarz_implementation.hpp
- include/laplacian_solvers_postprocessing.hpp
- include/laplacian_solvers_sequential_implementations.hpp

7.5 Result_Struct Struct Reference

Data structure to store the final computational results computed by the solver.

```
#include <laplacian_solvers.hpp>
```

Public Attributes

- eigenMatrix X

  Matrix containing the X coordinates of the structured mesh nodes.
- eigenMatrix Y

  Matrix containing the Y coordinates of the structured mesh nodes.
- eigenMatrix u_h

  Computed numerical approximate solution.
- unsigned iterations

  Number of iterations actually performed.
- double iteration_residue

  Norm of the final residual at the last iterative step.
- bool valid = false

  Validity flag (set to true only on Rank 0 after gathering data).

7.5.1 Detailed Description

Data structure to store the final computational results computed by the solver.

7.5.2 Member Data Documentation

7.5.2.1 iteration_residue

```
double Result_Struct::iteration_residue
```

Norm of the final residual at the last iterative step.

7.5.2.2 iterations

```
unsigned Result_Struct::iterations
```

Number of iterations actually performed.

7.5.2.3 u_h

```
eigenMatrix Result_Struct::u_h
```

Computed numerical approximate solution.

7.5.2.4 valid

```
bool Result_Struct::valid = false
```

Validity flag (set to true only on Rank 0 after gathering data).

7.5.2.5 X

```
eigenMatrix Result_Struct::X
```

Matrix containing the X coordinates of the structured mesh nodes.

7.5.2.6 Y

```
eigenMatrix Result_Struct::Y
```

Matrix containing the Y coordinates of the structured mesh nodes.

The documentation for this struct was generated from the following file:

- include/laplacian_solvers.hpp

Chapter 8

File Documentation

8.1 include/laplacian_convergence_test.hpp File Reference

Convergence test suite for verifying solver accuracy and performance speedup.

```
#include "laplacian_solvers.hpp"
#include "laplacian_solvers_implementation.hpp"
#include <cmath>
#include <vector>
#include <chrono>
```

Include dependency graph for laplacian_convergence_test.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct laplacian_solvers::Convergence_Struct

  Stores collected benchmark metrics across multiple grid refinement levels.
- class laplacian_solvers::Convergence_Test< solver_type, boundary_condition, funcType >

  Benchmark coordinator running serial and parallel execution comparison groups.

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.1.1 Detailed Description

Convergence test suite for verifying solver accuracy and performance speedup.

8.2 laplacian_convergence_test.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_CONVERGENCE_TEST_HPP
00002 #define LAPLACIAN_CONVERGENCE_TEST_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include "laplacian_solvers_implementation.hpp"
00006 #include <cmath>
00007 #include <vector>
00008 #include <chrono>
00009
00015 namespace laplacian_solvers {
00016
00023     struct Convergence_Struct {
00024         std::vector<double> errors_serial;
00025         std::vector<double> errors_parallel;
00026         std::vector<unsigned> n;
00027         std::vector<double> iterations_serial;
00028         std::vector<double> iterations_parallel;
00029         std::vector<double> times_serial;
00030         std::vector<double> times_parallel;
00031         std::vector<double> speedups;
00032     };
00033
00043     template <SolverType solver_type, BoundaryCondition boundary_condition, typename funcType>
00044     class Convergence_Test {
00045
00046         private:
00047         Data_Struct<funcType> data_original;
00048         std::vector<unsigned> refinement_levels = {8, 16, 32, 64, 128};
00049
00053         double compute_error(const eigenMatrix& u_h, const eigenMatrix& X, const eigenMatrix& Y,
      funcType u_exact_lambda);
00054
00055         Convergence_Struct convergence_data_struct;
00056         bool is_test_done = false;
00057         int mpi_rank;
00058
00059         public:
00064         Convergence_Test(const Data_Struct<funcType>& data) : data_original(data) {
00065             // Initialize MPI rank for potential use in parallel mesh building or solver execution
00066             MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
00067         }
00068
00073         Convergence_Struct run_convergence_test();
00074
00079         void print_convergence_results() const;
00080
00081     };
00082
00083     template <SolverType solver_type, BoundaryCondition boundary_condition, typename funcType>
00084     Convergence_Struct
      Convergence_Test<solver_type, boundary_condition, funcType>::run_convergence_test() {
00085
00086
00087         for(unsigned n_val : refinement_levels) {
00088             convergence_data_struct.n.push_back(n_val);
00089
00090             data_original.n = n_val;
00091
00092             // Serial Run
00093             auto start_serial = std::chrono::steady_clock::now();
00094
00095             Laplacian_Solver<solver_type, boundary_condition, ExecutionMode::SEQUENTIAL, funcType>
      solver_dirichlet_sequential(data_original);
00096             solver_dirichlet_sequential.build_mesh();
00097             auto result_serial = solver_dirichlet_sequential.solve();
00098
00099             auto end_serial = std::chrono::steady_clock::now();
00100
00101             std::chrono::duration<double, std::milli> elapsed_serial = end_serial - start_serial;
00102             convergence_data_struct.times_serial.push_back(elapsed_serial.count());
00103
00104
00105             double error_serial = compute_error(result_serial.u_h, result_serial.X, result_serial.Y,
      data_original.u_exact_lambda);
00106             convergence_data_struct.errors_serial.push_back(error_serial);
00107             convergence_data_struct.iterations_serial.push_back(result_serial.iterations);
00108         }
00109         unsigned i = 0;
00110         for(unsigned n_val : refinement_levels) {
00111             data_original.n = n_val;
00112
00113             // Parallel Run
```

```
00114             auto start_parallel = std::chrono::steady_clock::now();
00115
00116             Laplacian_Solver<solver_type, boundary_condition, ExecutionMode::PARALLEL, funcType>
      solver_dirichlet_parallel(data_original);
00117             solver_dirichlet_parallel.build_mesh();
00118             auto result_parallel = solver_dirichlet_parallel.solve();
00119
00120             auto end_parallel = std::chrono::steady_clock::now();
00121
00122             std::chrono::duration<double, std::milli> elapsed_parallel = end_parallel -
      start_parallel;
00123             convergence_data_struct.times_parallel.push_back(elapsed_parallel.count());
00124
00125             double error_parallel = compute_error(result_parallel.u_h, result_parallel.X,
      result_parallel.Y, data_original.u_exact_lambda);
00126             convergence_data_struct.errors_parallel.push_back(error_parallel);
00127             convergence_data_struct.iterations_parallel.push_back(result_parallel.iterations);
00128             convergence_data_struct.speedups.push_back(convergence_data_struct.times_serial[i] /
      convergence_data_struct.times_parallel[i]);
00129             i++;
00130         }
00131
00132         is_test_done = true;
00133         return convergence_data_struct;
00134     }
00135
00136     template <SolverType solver_type, BoundaryCondition boundary_condition, typename funcType>
00137     double Convergence_Test<solver_type, boundary_condition, funcType>::compute_error(const
      eigenMatrix& u_h, const eigenMatrix& X, const eigenMatrix& Y, funcType u_exact_lambda) {
00138         double error_sum = 0.0;
00139         unsigned n = u_h.rows();
00140         double h = (data_original.x2 - data_original.x1) / (n - 1);
00141         for (unsigned i = 0; i < n; ++i) {
00142             for (unsigned j = 0; j < n; ++j) {
00143                 double u_exact = u_exact_lambda(X(i, j), Y(i, j));
00144                 error_sum += h*std::pow(u_h(i, j) - u_exact, 2);
00145             }
00146         }
00147         return std::sqrt(error_sum);
00148
00149     }
00150
00151     template <SolverType solver_type, BoundaryCondition boundary_condition, typename funcType>
00152     void Convergence_Test<solver_type, boundary_condition, funcType>::print_convergence_results()
      const {
00153         if (mpi_rank != 0) {
00154             // Only the master process should print results to avoid clutter
00155             return;
00156         }
00157
00158         if (!is_test_done) {
00159             std::cerr « "Convergence test has not been run yet. Please run run_convergence_test()
      before printing results." « std::endl;
00160             return;
00161         }
00162         std::cout « "Refinement Level (n) | L2 Error (Serial) | L2 Error (Parallel) | Iterations
      (Serial) | Iterations (Parallel) | Time (ms, Serial) | Time (ms, Parallel) | Speedup" « std::endl;
00163         for (size_t i = 0; i < convergence_data_struct.n.size(); ++i) {
00164             std::cout « convergence_data_struct.n[i] « " | "
00165                       « convergence_data_struct.errors_serial[i] « " | "
00166                       « convergence_data_struct.errors_parallel[i] « " | "
00167                       « convergence_data_struct.iterations_serial[i] « " | "
00168                       « convergence_data_struct.iterations_parallel[i] « " | "
00169                       « convergence_data_struct.times_serial[i] « " | "
00170                       « convergence_data_struct.times_parallel[i] « " | "
00171                       « convergence_data_struct.speedups[i] « std::endl;
00172         }
00173         return;
00174     }
00175
00176 } // namespace laplacian_solvers
00177
00178 #endif // LAPLACIAN_CONVERGENCE_TEST_HPP
```

8.3 include/laplacian_solvers.hpp File Reference

Definition of the Laplacian/Poisson equation solver with multi-backend support (MPI, OpenMP, Sequential).

```
#include <iostream>
#include <cmath>
```

```
#include <vector>
#include <fstream>
#include <Eigen/Dense>
#include <chrono>
#include <mpi.h>
#include <omp.h>
#include <algorithm>
#include "laplacian_solvers_implementation.hpp"
#include "laplacian_solvers_postprocessing.hpp"
#include "laplacian_solvers_initializer.hpp"
#include "laplacian_solvers_sequential_implementations.hpp"
#include "laplacian_solvers_parallel_jacobi_implementation.hpp"
#include "laplacian_solvers_parallel_schwarz_implementation.hpp"
```

Include dependency graph for laplacian_solvers.hpp: This graph shows which files directly or indirectly include this file:

Classes

- struct Data_Struct< funcType >

  Holds all geometric, physical, and numerical configuration parameters for the problem.
- struct Result_Struct

  Data structure to store the final computational results computed by the solver.
- class laplacian_solvers::Laplacian_Solver< solver_type, boundary_condition, execution_mode, funcType >

  Core class for solving the Laplace/Poisson equation on structured grids.

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

Macros

- #define MAX_SCHWARZ_ITERATIONS 10

Typedefs

- using eigenMatrix = Eigen::Matrix< double, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor >

  Alias for Eigen matrices optimized in RowMajor to ensure better memory alignment during parallel access.

Enumerations

- enum class SolverType { JACOBI , SCHWARZ }

  Specifies the iterative numerical algorithm used to solve the linear system.
- enum class BoundaryCondition { DIRICHLET , NEUMANN , ROBIN }

  Specifies the physical boundary condition type applied to the domain edges.
- enum class ExecutionMode { SEQUENTIAL , PARALLEL }

  Specifies the execution policy of the solver (sequential or parallel).

8.3.1 Detailed Description

Definition of the Laplacian/Poisson equation solver with multi-backend support (MPI, OpenMP, Sequential).

8.3.2 Macro Definition Documentation

8.3.2.1 MAX_SCHWARZ_ITERATIONS

```
#define MAX_SCHWARZ_ITERATIONS 10
```

8.3.3 Typedef Documentation

8.3.3.1 eigenMatrix

```
using eigenMatrix = Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor>
```

Alias for Eigen matrices optimized in RowMajor to ensure better memory alignment during parallel access.

8.3.4 Enumeration Type Documentation

8.3.4.1 BoundaryCondition

```
enum class BoundaryCondition  [strong]
```

Specifies the physical boundary condition type applied to the domain edges.

Enumerator

DIRICHLET	Prescribed value on the boundary.
NEUMANN	Prescribed flux (normal derivative) on the boundary.
ROBIN	Mixed condition (linear combination of value and normal derivative).

8.3.4.2 ExecutionMode

```
enum class ExecutionMode  [strong]
```

Specifies the execution policy of the solver (sequential or parallel).

Enumerator

SEQUENTIAL	Sequential execution policy.
PARALLEL	Parallel execution policy using MPI and OpenMP.

8.3.4.3 SolverType

```
enum class SolverType  [strong]
```

Specifies the iterative numerical algorithm used to solve the linear system.

Enumerator

JACOBI	Jacobi iterative method.
SCHWARZ	Schwarz iterative method.

8.4 laplacian_solvers.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_HPP
00002 #define LAPLACIAN_SOLVERS_HPP
00003
00004 #include <iostream>
00005 #include <cmath>
00006 #include <vector>
00007 #include <fstream>
00008 #include <Eigen/Dense>
00009 #include <chrono>
00010 #include <mpi.h>
00011 #include <omp.h>
00012 #include <algorithm>
00013
00020 using eigenMatrix = Eigen::Matrix<double, Eigen::Dynamic, Eigen::Dynamic, Eigen::RowMajor>;
00021
00022 #define MAX_SCHWARZ_ITERATIONS 10
00023
00028 enum class SolverType {
00029     JACOBI,
00030     SCHWARZ
00031 };
00032
00037 enum class BoundaryCondition {
00038     DIRICHLET,
00039     NEUMANN,
00040     ROBIN
00041 };
00042
00047 enum class ExecutionMode {
00048     SEQUENTIAL,
00049     PARALLEL
00050 };
00051
00057 template <typename funcType>
00058 struct Data_Struct {
00059     unsigned n;
00060     double x1, x2;
00061
00062     funcType f0;
00063     funcType f1;
00064     funcType f2;
00065     funcType f3;
00066     funcType f4;
00067     funcType u_exact_lambda;
00068     double tolerance;
00069     unsigned max_iterations;
00070     double alpha;
00071     unsigned max_schwarz_iterations = 10;
00072 };
00073
00078 struct Result_Struct {
00079     eigenMatrix X;
00080     eigenMatrix Y;
00081     eigenMatrix u_h;
00082     unsigned iterations;
00083     double iteration_residue;
00084     bool valid = false;
00085 };
00086
```

```
00091 namespace laplacian_solvers {
00092
00103     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00104     class Laplacian_Solver {
00105         private:
00106             Data_Struct<funcType> data;
00107             eigenMatrix meshX;
00108             eigenMatrix meshY;
00109             double h;
00110             eigenMatrix u_exact;
00111             Result_Struct result;
00112
00113             int mpi_rank;
00114             int mpi_size;
00115             int mpi_used_size;
00116             bool participating = true;
00117
00118
00119         public:
00120
00121
00122             Laplacian_Solver(const Data_Struct<funcType>& d); //OK
00123
00124
00125             Result_Struct solve(); //OK
00126
00127
00128             void print() const;
00129
00130
00131             void export_to_vtk(const std::string& filename) const;
00132
00133             void build_mesh(); //OK
00134
00135             private:
00136             // PRIVATE HELPER METHODS
00137
00138             // INITIALIZERS
00139
00140             void build_mesh_sequential(); //OK
00141             void build_mesh_parallel(); //OK
00142             void process_filter();
00143
00144             void build_exact_solution(); //OK
00145             void build_exact_solution_sequential(); //OK
00146             void build_exact_solution_parallel(); //OK
00147
00148             // PRINT AND POSTPROCESSING
00149             void print_mesh() const; //OK
00150             void print_solution() const; //OK
00151             void print_exact_solution() const; //OK
00152
00153             //SOLVER STRUCTURE: sequential / parallel -> jacobi / schwarz -> dirichlet / neumann /
      robin
00154             Result_Struct parallel_solve(); //OK
00155
00156             Result_Struct jacobi_sequential(); //OK
00157             Result_Struct jacobi_parallel(); //OK
00158             Result_Struct schwarz_parallel(); //OK
00159
00160
00161     };
00162
00163 } // namespace laplacian_solvers
00164
00165 #include "laplacian_solvers_implementation.hpp"
00166 #include "laplacian_solvers_postprocessing.hpp"
00167 #include "laplacian_solvers_initializer.hpp"
00168
00169 #include "laplacian_solvers_sequential_implementations.hpp"
00170 #include "laplacian_solvers_parallel_jacobi_implementation.hpp"
00171 #include "laplacian_solvers_parallel_schwarz_implementation.hpp"
00172
00173
00174 #endif
```

8.5 include/laplacian_solvers_boundary_conditions.hpp File Reference

Functions to apply Dirichlet, Neumann, and Robin boundary conditions.

```
#include "laplacian_solvers.hpp"
```

Include dependency graph for laplacian_solvers_boundary_conditions.hpp: This graph shows which files directly or indirectly include this file:

Functions

- template<ExecutionMode execution_mode, typename funcType >
  void apply_dirichlet_condition (eigenMatrix &u_h, const Data_Struct< funcType > &data, const eigenMatrix &meshX, const eigenMatrix &meshY, const unsigned mpi_rank, const unsigned mpi_size)

  Applies Dirichlet boundary conditions (homogeneous or non-homogeneous).
- template<ExecutionMode execution_mode, typename funcType >
  void apply_neumann_condition (eigenMatrix &u_h, const Data_Struct< funcType > &data, const eigenMatrix &meshX, const eigenMatrix &meshY, const eigenMatrix &u_old, const unsigned mpi_rank, const unsigned mpi_size)

  Applies Neumann boundary conditions using a second-order central difference scheme.
- template<ExecutionMode execution_mode, typename funcType >
  void apply_robin_condition (eigenMatrix &u_h, const Data_Struct< funcType > &data, const eigenMatrix &meshX, const eigenMatrix &meshY, const eigenMatrix &u_old, const unsigned mpi_rank, const unsigned mpi_size)

  Applies Robin boundary conditions (mixed conditions matching value and normal derivative).
- template<BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType >
  void apply_boundary_condition (eigenMatrix &u_h, const Data_Struct< funcType > &data, const eigenMatrix &meshX, const eigenMatrix &meshY, const eigenMatrix &u_old={}, unsigned mpi_rank=0, unsigned mpi_↩ size=1)

  Interface function to route execution to the selected boundary condition type.

8.5.1 Detailed Description

Functions to apply Dirichlet, Neumann, and Robin boundary conditions.

8.5.2 Function Documentation

8.5.2.1 apply_boundary_condition()

```
template<BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType
>
void apply_boundary_condition (
            eigenMatrix & u_h,
            const Data_Struct< funcType > & data,
            const eigenMatrix & meshX,
            const eigenMatrix & meshY,
            const eigenMatrix & u_old = {},
            unsigned mpi_rank = 0,
            unsigned mpi_size = 1 )
```

Interface function to route execution to the selected boundary condition type.

Template Parameters

boundary_condition	Dirichlet, Neumann, or Robin selector.
execution_mode	Sequential or parallel policy backend.
funcType	Callable type for boundary and source terms.

Parameters

u_h	Solution matrix to update.
data	Struct containing configuration settings and data callbacks.
meshX	Matrix of X grid coordinates.
meshY	Matrix of Y grid coordinates.
u_old	Previous iteration solution matrix (default is empty for Dirichlet).
mpi_rank	Rank of the current MPI process (default 0).
mpi_size	Total number of MPI processes (default 1).

8.5.2.2 apply_dirichlet_condition()

```
template<ExecutionMode execution_mode, typename funcType >
void apply_dirichlet_condition (
             eigenMatrix & u_h,
             const Data_Struct< funcType > & data,
             const eigenMatrix & meshX,
             const eigenMatrix & meshY,
             const unsigned mpi_rank,
             const unsigned mpi_size )
```

Applies Dirichlet boundary conditions (homogeneous or non-homogeneous).

Template Parameters

execution_mode	Sequential or parallel execution policy.
funcType	Callable type for boundary functions.

Parameters

u_h	Matrix containing the current solution to update.
data	Struct containing domain and boundary function definitions.
meshX	Matrix of X grid coordinates.
meshY	Matrix of Y grid coordinates.
mpi_rank	Rank of the current MPI process.
mpi_size	Total number of MPI processes.

Here is the caller graph for this function:

8.5.2.3 apply_neumann_condition()

```
template<ExecutionMode execution_mode, typename funcType >
void apply_neumann_condition (
             eigenMatrix & u_h,
             const Data_Struct< funcType > & data,
             const eigenMatrix & meshX,
             const eigenMatrix & meshY,
             const eigenMatrix & u_old,
```

```
            const unsigned mpi_rank,
            const unsigned mpi_size )
```

Applies Neumann boundary conditions using a second-order central difference scheme.

Template Parameters

execution_mode	Sequential or parallel execution policy.
funcType	Callable type for boundary functions.

Parameters

u_h	Matrix containing the solution to update.
data	Struct containing domain, source term, and flux definitions.
meshX	Matrix of X grid coordinates.
meshY	Matrix of Y grid coordinates.
u_old	Solution matrix from the previous iteration step.
mpi_rank	Rank of the current MPI process.
mpi_size	Total number of MPI processes.

Note

Compatibility condition for pure Neumann problems is not checked by this function.

8.5.2.4 apply_robin_condition()

```
template<ExecutionMode execution_mode, typename funcType >
void apply_robin_condition (
            eigenMatrix & u_h,
            const Data_Struct< funcType > & data,
            const eigenMatrix & meshX,
            const eigenMatrix & meshY,
            const eigenMatrix & u_old,
            const unsigned mpi_rank,
            const unsigned mpi_size )
```

Applies Robin boundary conditions (mixed conditions matching value and normal derivative).

Template Parameters

execution_mode	Sequential or parallel execution policy.
funcType	Callable type for boundary functions.

Parameters

u_h	Matrix containing the solution to update.
data	Struct containing domain, source term, alpha coefficient, and boundary functions.
meshX	Matrix of X grid coordinates.
meshY	Matrix of Y grid coordinates.

Parameters

u_old	Solution matrix from the previous iteration step.
mpi_rank	Rank of the current MPI process.
mpi_size	Total number of MPI processes.

8.6 laplacian_solvers_boundary_conditions.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_BOUNDARY_CONDITIONS_HPP
00002 #define LAPLACIAN_SOLVERS_BOUNDARY_CONDITIONS_HPP
00003 #include "laplacian_solvers.hpp"
00004
00021 template <ExecutionMode execution_mode, typename funcType>
00022 void apply_dirichlet_condition(eigenMatrix & u_h, const Data_Struct<funcType>& data, const eigenMatrix
      & meshX, const eigenMatrix & meshY, const unsigned mpi_rank, const unsigned mpi_size){
00023
00024
00025     const unsigned last = data.n - 1;
00026
00027     if constexpr(execution_mode == ExecutionMode::SEQUENTIAL){
00028         // Sequential execution: mesh and u_h share the exact same global dimensions
00029         for(unsigned i = 0; i < data.n; i++){
00030             u_h(i, 0) = data.f4(meshX(i, 0), meshY(i, 0)); // Left edge
00031             u_h(i, last) = data.f2(meshX(i, last), meshY(i, last)); // Right edge
00032             u_h(0, i) = data.f1(meshX(0, i), meshY(0, i)); // Bottom edge
00033             u_h(last, i) = data.f3(meshX(last, i), meshY(last, i)); // Top edge
00034         }
00035
00036     } else if constexpr(execution_mode == ExecutionMode::PARALLEL){
00037
00038         const unsigned up_row = (mpi_rank == 0 ? 0 : 1);
00039         const unsigned down_row = (mpi_rank == mpi_size - 1 ? 0 : 1);
00040
00041         //const unsigned real_subdomain_rows = u_h.rows() - up_row - down_row;
00042
00043         // Bottom side
00044         if(mpi_rank == 0) {
00045             for(unsigned i = 0; i < data.n; i++) {
00046                 u_h(0, i) = data.f1(meshX(0, i), meshY(0, i));
00047             }
00048         }
00049
00050         // Top side
00051         if(mpi_rank == mpi_size - 1) {
00052             const unsigned local_last_row = u_h.rows() - 1;
00053             const unsigned mesh_last_row = meshY.rows() - 1;
00054             for(unsigned i = 0; i < data.n; i++) {
00055                 u_h(local_last_row, i) = data.f3(meshX(mesh_last_row, i), meshY(mesh_last_row, i));
00056             }
00057         }
00058
00059         // Left - Right sides (
00060         const unsigned start_loop = up_row;
00061         const unsigned end_loop = u_h.rows() - down_row;
00062
00063         for(unsigned i = start_loop; i < end_loop; i++){
00064             unsigned mesh_i = i - up_row;
00065             u_h(i, 0)    = data.f4(meshX(mesh_i, 0), meshY(mesh_i, 0));
00066             u_h(i, last) = data.f2(meshX(mesh_i, last), meshY(mesh_i, last));
00067         }
00068     }
00069 }
00070
00084 template <ExecutionMode execution_mode, typename funcType>
00085 void apply_neumann_condition(eigenMatrix& u_h, const Data_Struct<funcType>& data, const eigenMatrix&
      meshX, const eigenMatrix& meshY, const eigenMatrix& u_old, const unsigned mpi_rank, const unsigned
      mpi_size){
00086     const unsigned last = data.n - 1;
00087     const double h  = std::abs(data.x2 - data.x1) / (data.n - 1);
00088     const double h2 = h * h;
00089
00090     if constexpr (execution_mode == ExecutionMode::SEQUENTIAL) {
00091
00092         for (unsigned i = 1; i < last; i++) {
00093             // Bottom side
```

```
00094             u_h(0, i) = 0.25*(2*u_old(1,i) + u_old(0,i-1) + u_old(0,i+1) + h2*data.f0(meshX(0,i),
      meshY(0,i)) + 2*h*data.f1(meshX(0,i), meshY(0,i)));
00095             // Top side
00096             u_h(last, i) = 0.25*(2*u_old(last-1,i) + u_old(last,i-1) + u_old(last,i+1) +
      h2*data.f0(meshX(last,i), meshY(last,i)) + 2*h*data.f3(meshX(last,i), meshY(last,i)));
00097             // Left side
00098             u_h(i, 0) = 0.25*(u_old(i-1,0) + u_old(i+1,0) + 2*u_old(i,1) + h2*data.f0(meshX(i,0),
      meshY(i,0)) + 2*h*data.f4(meshX(i,0), meshY(i,0)));
00099             // Right side
00100             u_h(i, last) = 0.25*(u_old(i-1,last) + u_old(i+1,last) + 2*u_old(i,last-1) +
      h2*data.f0(meshX(i,last), meshY(i,last)) + 2*h*data.f2(meshX(i,last), meshY(i,last)));
00101         }
00102
00103         // Corners
00104         u_h(0, 0) = 0.25*(2*u_old(1,0) + 2*u_old(0,1) + h2*data.f0(meshX(0,0), meshY(0,0)) +
      2*h*(data.f1(meshX(0,0), meshY(0,0)) + data.f4(meshX(0,0), meshY(0,0))));
00105         u_h(0, last) = 0.25*(2*u_old(1,last) + 2*u_old(0,last-1) + h2*data.f0(meshX(0,last),
      meshY(0,last)) + 2*h*(data.f1(meshX(0,last), meshY(0,last)) + data.f2(meshX(0,last), meshY(0,last))));
00106         u_h(last, 0) = 0.25*(2*u_old(last-1,0) + 2*u_old(last,1) + h2*data.f0(meshX(last,0),
      meshY(last,0)) + 2*h*(data.f3(meshX(last,0), meshY(last,0)) + data.f4(meshX(last,0), meshY(last,0))));
00107         u_h(last, last) = 0.25*(2*u_old(last-1,last) + 2*u_old(last,last-1) +
      h2*data.f0(meshX(last,last), meshY(last,last)) + 2*h*(data.f3(meshX(last,last), meshY(last,last)) +
      data.f2(meshX(last,last), meshY(last,last))));
00108
00109     } else if constexpr (execution_mode == ExecutionMode::PARALLEL) {
00110
00111         // Compute indexes
00112         const unsigned up_row = (mpi_rank == 0)? 0 : 1;
00113         const unsigned down_row = (mpi_rank == mpi_size - 1) ? 0 : 1;
00114         const unsigned real_first = up_row;
00115         const unsigned real_last = u_h.rows() - 1 - down_row;
00116
00117         // Left - right sides without the corners
00118         for (unsigned i = real_first; i <= real_last; i++) {
00119             const bool is_global_bottom = (mpi_rank == 0 && i == real_first);
00120             const bool is_global_top = (mpi_rank == mpi_size - 1 && i == real_last);
00121             if (is_global_bottom || is_global_top) continue;
00122
00123             const unsigned mi = i - up_row;
00124             u_h(i, 0) = 0.25*(u_old(i-1,0) + u_old(i+1,0) + 2*u_old(i,1) + h2*data.f0(meshX(mi,0),
      meshY(mi,0)) + 2*h*data.f4(meshX(mi,0), meshY(mi,0)));
00125             u_h(i, last) = 0.25*(u_old(i-1,last)  + u_old(i+1,last) + 2*u_old(i,last-1) +
      h2*data.f0(meshX(mi,last), meshY(mi,last)) + 2*h*data.f2(meshX(mi,last), meshY(mi,last)));
00126         }
00127
00128         // Bottom side + corner
00129         if (mpi_rank == 0) {
00130             for (unsigned j = 1; j < last; j++)
00131                 u_h(0, j) = 0.25*(2*u_old(1,j) + u_old(0,j-1) + u_old(0,j+1) + h2*data.f0(meshX(0,j),
      meshY(0,j)) + 2*h*data.f1(meshX(0,j), meshY(0,j)));
00132
00133             u_h(0, 0) = 0.25*(2*u_old(1,0) + 2*u_old(0,1) + h2*data.f0(meshX(0,0), meshY(0,0)) +
      2*h*(data.f1(meshX(0,0), meshY(0,0)) + data.f4(meshX(0,0), meshY(0,0))));
00134             u_h(0, last) = 0.25*(2*u_old(1,last) + 2*u_old(0,last-1) + h2*data.f0(meshX(0,last),
      meshY(0,last)) + 2*h*(data.f1(meshX(0,last), meshY(0,last)) + data.f2(meshX(0,last), meshY(0,last))));
00135         }
00136
00137         // Top side + corner
00138         if (mpi_rank == mpi_size - 1) {
00139             const unsigned mt = real_last - up_row;
00140             for (unsigned j = 1; j < last; j++)
00141                 u_h(real_last, j) = 0.25*(2*u_old(real_last-1,j) + u_old(real_last,j-1) +
      u_old(real_last,j+1) + h2*data.f0(meshX(mt,j), meshY(mt,j)) + 2*h*data.f3(meshX(mt,j), meshY(mt,j)));
00142
00143             u_h(real_last, 0) = 0.25*(2*u_old(real_last-1,0) + 2*u_old(real_last,1) +
      h2*data.f0(meshX(mt,0), meshY(mt,0))+ 2*h*(data.f3(meshX(mt,0), meshY(mt,0)) + data.f4(meshX(mt,0),
      meshY(mt,0))));
00144             u_h(real_last, last) = 0.25*(2*u_old(real_last-1,last) + 2*u_old(real_last,last-1) +
      h2*data.f0(meshX(mt,last), meshY(mt,last)) + 2*h*(data.f3(meshX(mt,last), meshY(mt,last)) +
      data.f2(meshX(mt,last), meshY(mt,last))));
00145         }
00146     }
00147 }
00148
00161 template <ExecutionMode execution_mode, typename funcType>
00162 void apply_robin_condition(eigenMatrix & u_h, const Data_Struct<funcType>& data, const eigenMatrix &
      meshX, const eigenMatrix& meshY, const eigenMatrix & u_old, const unsigned mpi_rank, const unsigned
      mpi_size){
00163
00164     // General variables
00165     const unsigned last = data.n - 1;
00166     const double h = std::abs(data.x2 - data.x1) / (data.n - 1);
00167     const double h2 = h * h;
00168
00169     // Denominators
00170     const double den_edge = 4.0 + 2.0 * h * data.alpha;
00171     const double den_corner = 4.0 + 4.0 * h * data.alpha;
```

```
00172
00173     if constexpr(execution_mode == ExecutionMode::SEQUENTIAL){
00174         // Left - right sides without the corners
00175         for (unsigned i = 1; i < last; ++i) {
00176             u_h(i, 0)    = (u_old(i-1, 0) + u_old(i+1, 0) + 2.0 * u_old(i, 1) + h2 * data.f0(meshX(i,
      0), meshY(i, 0)) + 2.0 * h * data.f4(meshX(i, 0), meshY(i, 0))) / den_edge;
00177             u_h(i, last) = (u_old(i-1, last) + u_old(i+1, last) + 2.0 * u_old(i, last-1) + h2 *
      data.f0(meshX(i, last), meshY(i, last)) + 2.0 * h * data.f2(meshX(i, last), meshY(i, last))) /
      den_edge;
00178         }
00179
00180         // Bottom - top sides without the corners
00181         for (unsigned j = 1; j < last; ++j) {
00182             u_h(0, j)    = (2.0 * u_old(1, j) + u_old(0, j-1) + u_old(0, j+1) + h2 * data.f0(meshX(0,
      j), meshY(0, j)) + 2.0 * h * data.f1(meshX(0, j), meshY(0, j))) / den_edge;
00183             u_h(last, j) = (2.0 * u_old(last-1, j) + u_old(last, j-1) + u_old(last, j+1) + h2 *
      data.f0(meshX(last, j), meshY(last, j)) + 2.0 * h * data.f3(meshX(last, j), meshY(last, j))) /
      den_edge;
00184         }
00185
00186         // Corners
00187         u_h(0, 0) = (2.0 * u_old(1, 0) + 2.0 * u_old(0, 1) + h2 * data.f0(meshX(0,0), meshY(0,0)) +
      2.0 * h * (data.f1(meshX(0,0), meshY(0,0)) + data.f4(meshX(0,0), meshY(0,0)))) / den_corner;
00188         u_h(0, last) = (2.0 * u_old(1, last) + 2.0 * u_old(0, last-1) + h2 * data.f0(meshX(0,last),
      meshY(0,last)) + 2.0 * h * (data.f1(meshX(0,last), meshY(0,last)) + data.f2(meshX(0,last),
      meshY(0,last)))) / den_corner;
00189         u_h(last, 0) = (2.0 * u_old(last-1, 0) + 2.0 * u_old(last, 1) + h2 * data.f0(meshX(last,0),
      meshY(last,0)) + 2.0 * h * (data.f3(meshX(last,0), meshY(last,0)) + data.f4(meshX(last,0),
      meshY(last,0)))) / den_corner;
00190         u_h(last, last) = (2.0 * u_old(last-1, last) + 2.0 * u_old(last, last-1) + h2 *
      data.f0(meshX(last,last), meshY(last,last)) + 2.0 * h * (data.f3(meshX(last,last), meshY(last,last)) +
      data.f2(meshX(last,last), meshY(last,last)))) / den_corner;
00191     }
00192
00193     if constexpr(execution_mode == ExecutionMode::PARALLEL){
00194
00195         // Compute indexes
00196         const unsigned remainder_rows = data.n % mpi_size;
00197         const unsigned local_rows = data.n / mpi_size + (mpi_rank < remainder_rows ? 1 : 0);
00198
00199         const unsigned up_row = (mpi_rank == 0 ? 0 : 1);
00200         const unsigned down_row = (mpi_rank == mpi_size - 1 ? 0 : 1);
00201
00202         const unsigned first_real_row = up_row;
00203         const unsigned last_real_row = u_h.rows() - 1 - down_row;
00204
00205         // Left - right sides without corners
00206         for(unsigned i = first_real_row; i <= last_real_row; i++){
00207             const bool is_global_bottom = (mpi_rank == 0 && i == first_real_row);
00208             const bool is_global_top    = (mpi_rank == mpi_size - 1 && i == last_real_row);
00209             if (is_global_bottom || is_global_top) continue;
00210
00211             unsigned mesh_i = i - up_row;
00212
00213             u_h(i, 0)    = (u_old(i-1, 0) + u_old(i+1, 0) + 2.0 * u_old(i, 1) + h2 *
      data.f0(meshX(mesh_i, 0), meshY(mesh_i, 0)) + 2.0 * h * data.f4(meshX(mesh_i, 0), meshY(mesh_i, 0))) /
      den_edge;
00214             u_h(i, last) = (u_old(i-1, last) + u_old(i+1, last) + 2.0 * u_old(i, last-1) + h2 *
      data.f0(meshX(mesh_i, last), meshY(mesh_i, last)) + 2.0 * h * data.f2(meshX(mesh_i, last),
      meshY(mesh_i, last))) / den_edge;
00215         }
00216
00217
00218         if(mpi_rank == 0) {
00219             // Bottom side
00220             for(unsigned j = 1; j < last; j++) u_h(0, j) = (2.0 * u_old(1, j) + u_old(0, j-1) +
      u_old(0, j+1) + h2 * data.f0(meshX(0, j), meshY(0, j)) + 2.0 * h * data.f1(meshX(0, j), meshY(0, j)))
      / den_edge;
00221             // Left - bottom corner
00222             u_h(0, 0) = (2.0 * u_old(1, 0) + 2.0 * u_old(0, 1) + h2 * data.f0(meshX(0,0), meshY(0,0))
      + 2.0 * h * (data.f1(meshX(0,0), meshY(0,0)) + data.f4(meshX(0,0), meshY(0,0)))) / den_corner;
00223             // Right - bottom corner
00224             u_h(0, last) = (2.0 * u_old(1, last) + 2.0 * u_old(0, last-1) + h2 *
      data.f0(meshX(0,last), meshY(0,last)) + 2.0 * h * (data.f1(meshX(0,last), meshY(0,last)) +
      data.f2(meshX(0,last), meshY(0,last)))) / den_corner;
00225         }
00226
00227         // Top side
00228         if(mpi_rank == mpi_size - 1) {
00229             const unsigned local_last_row = u_h.rows() - 1;
00230             const unsigned mesh_last_row = local_rows - 1;
00231
00232             // Top side
00233             for(unsigned j = 1; j < last; j++) u_h(local_last_row, j) = (2.0 * u_old(local_last_row -
      1, j) + u_old(local_last_row, j-1) + u_old(local_last_row, j+1) + h2 * data.f0(meshX(mesh_last_row,
      j), meshY(mesh_last_row, j)) + 2.0 * h * data.f3(meshX(mesh_last_row, j), meshY(mesh_last_row, j))) /
      den_edge;
```

```
00234             // Left - top corner
00235             u_h(local_last_row, 0) = (2.0 * u_old(local_last_row-1, 0) + 2.0 * u_old(local_last_row,
      1) + h2 * data.f0(meshX(mesh_last_row,0), meshY(mesh_last_row,0)) + 2.0 * h *
      (data.f3(meshX(mesh_last_row,0), meshY(mesh_last_row,0)) + data.f4(meshX(mesh_last_row,0),
      meshY(mesh_last_row,0)))) / den_corner;
00236             // Right - top corner
00237             u_h(local_last_row, last) = (2.0 * u_old(local_last_row-1, last) + 2.0 *
      u_old(local_last_row, last-1) + h2 * data.f0(meshX(mesh_last_row,last), meshY(mesh_last_row,last)) +
      2.0 * h * (data.f3(meshX(mesh_last_row,last), meshY(mesh_last_row,last)) +
      data.f2(meshX(mesh_last_row,last), meshY(mesh_last_row,last)))) / den_corner;
00238         }
00239     }
00240 }
00241
00255 template <BoundaryCondition boundary_condition, ExecutionMode execution_mode, typename funcType>
00256 void apply_boundary_condition(eigenMatrix & u_h, const Data_Struct<funcType>& data, const eigenMatrix
      & meshX, const eigenMatrix &meshY, const eigenMatrix & u_old = {}, unsigned mpi_rank = 0, unsigned
      mpi_size = 1){
00257     if constexpr(boundary_condition == BoundaryCondition::DIRICHLET)
      apply_dirichlet_condition<execution_mode, funcType>(u_h, data, meshX, meshY, mpi_rank, mpi_size);
00258     if constexpr(boundary_condition == BoundaryCondition::NEUMANN)
      apply_neumann_condition<execution_mode, funcType>(u_h, data, meshX, meshY, u_old, mpi_rank, mpi_size);
00259     if constexpr(boundary_condition == BoundaryCondition::ROBIN)
      apply_robin_condition<execution_mode, funcType>(u_h, data, meshX, meshY, u_old, mpi_rank, mpi_size);
00260 }
00261
00262 #endif // LAPLACIAN_SOLVERS_BOUNDARY_CONDITIONS_HPP
```

8.7 include/laplacian_solvers_implementation.hpp File Reference

Dispatching logic for the sequential and parallel Laplacian solvers.

```
#include "laplacian_solvers.hpp"
#include "laplacian_solvers_boundary_conditions.hpp"
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>
```

Include dependency graph for laplacian_solvers_implementation.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.7.1 Detailed Description

Dispatching logic for the sequential and parallel Laplacian solvers.

8.8 laplacian_solvers_implementation.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_IMPLEMENTATION_HPP
00002 #define LAPLACIAN_SOLVERS_IMPLEMENTATION_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include "laplacian_solvers_boundary_conditions.hpp"
00006
00007 #include <iostream>
00008 #include <cmath>
```

```
00009 #include <vector>
00010 #include <fstream>
00011
00017 namespace laplacian_solvers{
00018
00031     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode,typename funcType>
00032     Result_Struct
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::solve(){
00033
00034         if(mpi_rank >= mpi_size) return result;
00035
00036         if constexpr (execution_mode == ExecutionMode::SEQUENTIAL) {
00037             if(mpi_rank != 0) return result; // Only rank 0 will run the sequential solver
00038             return jacobi_sequential();
00039         } else {
00040             return parallel_solve();
00041         }
00042
00043     }
00044
00055     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00056     Result_Struct
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::parallel_solve(){
00057         if constexpr (solver_type == SolverType::JACOBI) {
00058             return jacobi_parallel();
00059         } else {
00060             return schwarz_parallel();
00061         }
00062     }
00063
00064 }
00065
00066 #endif
```

8.9 include/laplacian_solvers_initializer.hpp File Reference

Implementation of initialization, mesh generation, and exact solution building for Laplacian_Solver.

```
#include <omp.h>
#include <mpi.h>
#include "laplacian_solvers.hpp"
```

Include dependency graph for laplacian_solvers_initializer.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.9.1 Detailed Description

Implementation of initialization, mesh generation, and exact solution building for Laplacian_Solver.

8.10 laplacian_solvers_initializer.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_INITIALIZER_HPP
00002 #define LAPLACIAN_SOLVERS_INITIALIZER_HPP
00003
00004 #include <omp.h>
00005 #include <mpi.h>
00006
00007 #include "laplacian_solvers.hpp"
00008
00014 namespace laplacian_solvers{
00015
00016     /* --- CONSTRUCTOR --- */
00017
00029     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode,
00030               typename funcType>
00031
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::Laplacian_Solver(const
      Data_Struct<funcType>& d) : data(d) {
00032
00033         // Controls process availability, builds mesh and exact solution
00034         process_filter();
00035         build_mesh();
00036         build_exact_solution();
00037
00038     };
00039
00046     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00047     inline void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::process_filter(){
00048
00049         MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
00050         MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
00051
00052         // If more processes are available than rows throw error
00053
00054         if constexpr(execution_mode == ExecutionMode::PARALLEL){
00055             if(mpi_size > static_cast<int>(data.n)) {
00056                 if (mpi_rank == 0) std::cerr « "Error: program launched with " « mpi_size « "
      processors, but number of rows is " « data.n « ". Aborting..." « std::endl;
00057                 MPI_Abort(MPI_COMM_WORLD, 1);
00058             }
00059         }
00060
00061     }
00062
00063     /* --- MESH GENERATION AND INITIALIZATION --- */
00064
00070     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode,
00071               typename funcType>
00072     void Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_mesh(){
00073
00074         h = abs(data.x2 - data.x1) / (data.n - 1); // Uniform grid spacing. Works even if x2 < x1.
00075
00076         if constexpr (execution_mode == ExecutionMode::SEQUENTIAL) build_mesh_sequential();
00077         else build_mesh_parallel();
00078     }
00079
00085     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00086     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_mesh_sequential(){
00087
00088         meshX = eigenMatrix::Zero(data.n, data.n);
00089         meshY = eigenMatrix::Zero(data.n, data.n);
00090
00091         for(unsigned i = 0; i < data.n; i++) for(unsigned j = 0; j < data.n; j++) {
00092             meshX(i, j) = data.x1 + j * h;
00093             meshY(i, j) = data.x1 + i * h;
00094         }
00095     }
00096
00104     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00105     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_mesh_parallel(){
00106
00107         // Get rank information
00108
00109         // Distribute rows among processes. If the number of rows is not divided by the number of
      threads, the remaining ones will be assigned to the
```

```
00110         // fisrt threads, as shown in Figure 1 of the challenge pdf file.
00111         const unsigned local_rows = data.n / mpi_size;
00112         const unsigned remainder_rows = data.n % mpi_size;
00113 
00114         const unsigned start_row = mpi_rank * local_rows + std::min(remainder_rows,
      static_cast<unsigned>(mpi_rank));
00115         const unsigned end_row = start_row + local_rows + (static_cast<unsigned>(mpi_rank) <
      remainder_rows ? 1 : 0);
00116 
00117         meshX = eigenMatrix::Zero(end_row - start_row, data.n);
00118         meshY = eigenMatrix::Zero(end_row - start_row, data.n);
00119 
00120         #pragma omp parallel for
00121         for(unsigned i = 0; i < end_row - start_row; i++) for(unsigned j = 0; j < data.n; j++){
00122             meshX(i, j) = data.x1 + j * h;
00123             meshY(i, j) = data.x1 + (i + start_row) * h;
00124         }
00125     }
00126 
00130     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00131     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_exact_solution(){
00132 
00133         u_exact = eigenMatrix::Zero(data.n, data.n);
00134 
00135         if constexpr (execution_mode == ExecutionMode::SEQUENTIAL) build_exact_solution_sequential();
00136         else build_exact_solution_parallel();
00137     }
00138 
00142     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00143     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_exact_solution_sequential(){
00144 
00145         for(unsigned i = 0; i < data.n; i++) for(unsigned j = 0; j < data.n; j++)
00146             u_exact(i, j) = data.u_exact_lambda(meshX(i, j), meshY(i, j));
00147 
00148     }
00149 
00155     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00156     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::build_exact_solution_parallel(){
00157 
00158 
00159         // Distribute rows among processes. If the number of rows is not divided by the number of
      threads, the remaining ones will be assigned to the
00160         // fisrt threads, as shown in Figure 1 of the challenge pdf file.
00161         const unsigned local_rows = meshX.rows();
00162 
00163         u_exact = eigenMatrix::Zero(local_rows, data.n);
00164 
00165         #pragma omp parallel for
00166         for(unsigned i = 0; i < local_rows; i++) for(unsigned j = 0; j < data.n; j++){
00167             u_exact(i, j) = data.u_exact_lambda(meshX(i, j), meshY(i, j));
00168         }
00169 
00170     }
00171 }// namespace laplacian_solvers
00172 
00173 #endif // LAPLACIAN_SOLVERS_INITIALIZER_HPP
```

8.11 include/laplacian_solvers_parallel_jacobi_implementation.hpp File Reference

Hybrid MPI+OpenMP parallel implementation of the Jacobi solver.

```
#include "laplacian_solvers.hpp"
#include "laplacian_solvers_boundary_conditions.hpp"
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>
```

Include dependency graph for laplacian_solvers_parallel_jacobi_implementation.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.11.1 Detailed Description

Hybrid MPI+OpenMP parallel implementation of the Jacobi solver.

8.12 laplacian_solvers_parallel_jacobi_implementation.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_PARALLEL_JACOBI_IMPLEMENTATION_HPP
00002 #define LAPLACIAN_SOLVERS_PARALLEL_JACOBI_IMPLEMENTATION_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include "laplacian_solvers_boundary_conditions.hpp"
00006
00007 #include <iostream>
00008 #include <cmath>
00009 #include <vector>
00010 #include <fstream>
00011
00017 namespace laplacian_solvers{
00018
00030     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00031     Result_Struct
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::jacobi_parallel(){
00032
00033         // Compute indexes and communication info
00034         const int remainder_rows = static_cast<int>(data.n) % mpi_size;
00035         const int local_rows = static_cast<int>(data.n) / mpi_size + (mpi_rank < remainder_rows ? 1 :
      0);
00036
00037         // Find process neighbors
00038         const int rank_up = (mpi_rank == 0) ? MPI_PROC_NULL : mpi_rank - 1;
00039         const int rank_down = (mpi_rank == mpi_size - 1) ? MPI_PROC_NULL : mpi_rank + 1;
00040
00041         const int up_row = (mpi_rank == 0 ? 0 : 1);
00042         const int down_row = (mpi_rank == mpi_size - 1 ? 0 : 1);
00043         const int total_rows = local_rows + up_row + down_row;
00044
00045         // Build local matrixes. It will also host the upper and lower rows for later communication
00046         eigenMatrix u_h_local = eigenMatrix::Zero(total_rows, data.n);
00047
00048         // Apply boundary contitions to the initial guess
00049         if constexpr(boundary_condition == BoundaryCondition::DIRICHLET || boundary_condition ==
      BoundaryCondition::ROBIN)
00050             apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_h_local, data,
      meshX, meshY, u_h_local, mpi_rank, mpi_size);
00051
00052         // Initialize loop
00053         eigenMatrix u_h_new_local(u_h_local);
00054         const double tol_squared = data.tolerance * data.tolerance;
00055         double err = tol_squared + 1.0;
00056         unsigned iter = 0;
00057         const double h2 = h * h;
00058         #pragma omp parallel
00059         {
00060             while(err > tol_squared && iter < data.max_iterations){
00061                 // First - Handle communication with other processes
00062                 #pragma omp single
00063                 {
00064                     MPI_Sendrecv(u_h_local.data() + up_row * data.n, data.n, MPI_DOUBLE, rank_up, 0,
00065                                  u_h_local.data() + (total_rows - 1) * data.n, data.n, MPI_DOUBLE,
      rank_down, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
00066
00067                     MPI_Sendrecv(u_h_local.data() + (total_rows - 1 - down_row) * data.n, data.n,
      MPI_DOUBLE, rank_down, 1,
00068                                  u_h_local.data(), data.n, MPI_DOUBLE, rank_up, 1, MPI_COMM_WORLD,
      MPI_STATUS_IGNORE);
00069                 }
00070
```

```
00071                 // Second - Compute new values, but not the received ones!
00072                 #pragma omp for collapse(2)
00073                 for(int i = 1; i < total_rows - 1; i++){
00074                     for(unsigned j = 1; j < data.n - 1; j++){
00075                         u_h_new_local(i, j) = 0.25 * (u_h_local(i-1, j) + u_h_local(i+1, j) +
      u_h_local(i, j-1) + u_h_local(i, j+1) + h2 * data.f0(meshX(i - up_row, j), meshY(i - up_row, j)));

00076                     }
00077                 }
00078
00079                 // Third - Apply boundary conditions
00080                 #pragma omp single
00081                 {
00082                     if constexpr(boundary_condition == BoundaryCondition::NEUMANN ||
      boundary_condition == BoundaryCondition::ROBIN)
00083
      apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_h_new_local, data, meshX,
      meshY, u_h_local, mpi_rank, mpi_size);
00084
00085                     // Fourth - zero-average constraint for Neumann BCs (required to ensure uniqueness
      of the solution)
00086                     if constexpr(boundary_condition == BoundaryCondition::NEUMANN){
00087                         double avg = 0.0;
00088                         const double local_sum = u_h_new_local.block(up_row, 0, local_rows,
      data.n).sum();
00089                         MPI_Allreduce(&local_sum, &avg, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
00090                         u_h_new_local.array() -= avg / (data.n * data.n);
00091                     }
00092
00093                     // Fourth - Compute local error in L2 norm (frobenius norm) with h
00094                     const double local_err = (u_h_new_local.block(up_row, 0, local_rows, data.n) -
      u_h_local.block(up_row, 0, local_rows, data.n)).squaredNorm();
00095
00096                     // Fifth - Compute global error and prepare next iteration
00097                     MPI_Allreduce(&local_err, &err, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
00098                     err = h * err; // Maybe it is possible to avoid the sqrt?
00099                     iter++;
00100                     u_h_local.swap(u_h_new_local);
00101                 } // single
00102             } // while
00103         } // parallel
00104
00105         // Prepare communication
00106         std::vector<int> recv_counts(mpi_size);
00107         std::vector<int> displs(mpi_size);
00108
00109         for(int i = 0; i < mpi_size; i++){
00110             recv_counts[i] = static_cast<int>(data.n) * (static_cast<int>(data.n) / mpi_size + (i <
      remainder_rows ? 1 : 0));
00111             displs[i] = (i == 0) ? 0 : displs[i-1] + recv_counts[i-1];
00112         }
00113
00114         // Assemble global solution and mesh
00115         eigenMatrix u_h_global(data.n, data.n); // Maybe initialize result.u_h?
00116         eigenMatrix meshX_global(data.n, data.n);
00117         eigenMatrix meshY_global(data.n, data.n);
00118
00119
00120         MPI_Gatherv(u_h_local.data() + up_row * data.n, local_rows * data.n, MPI_DOUBLE,
      u_h_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00121         MPI_Gatherv(meshX.data(), local_rows * data.n, MPI_DOUBLE, meshX_global.data(),
      recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00122         MPI_Gatherv(meshY.data(), local_rows * data.n, MPI_DOUBLE, meshY_global.data(),
      recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00123
00124         // Return results - caution: only rank 0 has the correct return structure!
00125         result.u_h.swap(u_h_global);
00126         result.X.swap(meshX_global);
00127         result.Y.swap(meshY_global);
00128         result.iterations = iter;
00129         result.iteration_residue = std::sqrt(err);
00130         if(mpi_rank == 0) result.valid = true;
00131
00132         return result;
00133
00134     }
00135 } // namespace laplacian_solvers
00136
00137
00138 #endif // LAPLACIAN_SOLVERS_PARALLEL_JACOBI_IMPLEMENTATION_HPP
```

8.13 include/laplacian_solvers_parallel_schwarz_implementation.hpp File Reference

Hybrid MPI+OpenMP parallel implementation of the Schwarz domain decomposition method.

```
#include "laplacian_solvers.hpp"
#include "laplacian_solvers_boundary_conditions.hpp"
#include <iostream>
#include <cmath>
#include <vector>
```

Include dependency graph for laplacian_solvers_parallel_schwarz_implementation.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

Macros

- #define MAX_SCHWARZ_ITERATIONS 10

8.13.1 Detailed Description

Hybrid MPI+OpenMP parallel implementation of the Schwarz domain decomposition method.

8.13.2 Macro Definition Documentation

8.13.2.1 MAX_SCHWARZ_ITERATIONS

```
#define MAX_SCHWARZ_ITERATIONS 10
```

8.14 laplacian_solvers_parallel_schwarz_implementation.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_PARALLEL_SCHWARZ_IMPLEMENTATION_HPP
00002 #define LAPLACIAN_SOLVERS_PARALLEL_SCHWARZ_IMPLEMENTATION_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include "laplacian_solvers_boundary_conditions.hpp"
00006 #include <iostream>
00007 #include <cmath>
00008 #include <vector>
00009
00015 #define MAX_SCHWARZ_ITERATIONS 10
00016
00017 namespace laplacian_solvers{
00018
00030     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00031     Result_Struct
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::schwarz_parallel(){
00032
00033         // Compute indexes and communication info
```

```
00034         const unsigned remainder_rows = data.n % mpi_size;
00035         const unsigned local_rows = data.n / mpi_size + (static_cast<unsigned>(mpi_rank) <
      remainder_rows ? 1 : 0);
00036 
00037         // Indexes for meshX and meshY
00038 
00039         // Find process neighbors
00040         const int rank_up = (mpi_rank == 0) ? MPI_PROC_NULL : mpi_rank - 1;
00041         const int rank_down = (mpi_rank == mpi_size - 1) ? MPI_PROC_NULL : mpi_rank + 1;
00042 
00043         const unsigned up_row = (mpi_rank == 0 ? 0 : 1);
00044         const unsigned down_row = (mpi_rank == mpi_size - 1 ? 0 : 1);
00045         const unsigned total_rows = local_rows + up_row + down_row;
00046 
00047         // Build local matrixes
00048         eigenMatrix u_h_local = eigenMatrix::Zero(total_rows, data.n);
00049 
00050         // Apply boundary conditions to the initial guess
00051         if constexpr(boundary_condition == BoundaryCondition::DIRICHLET || boundary_condition ==
      BoundaryCondition::ROBIN)
00052             apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_h_local, data,
      meshX, meshY, u_h_local, mpi_rank, mpi_size);
00053 
00054         // Initialize loop variables
00055         eigenMatrix u_h_new_local(u_h_local);
00056         eigenMatrix u_h_old_global_step(u_h_local);
00057         double global_err = data.tolerance + 1.0;
00058         unsigned global_iter = 0;
00059         const double h2 = h * h;
00060 
00061         // Global Schwarz iteration loop
00062 
00063         #pragma omp parallel
00064         {
00065             while(global_err > data.tolerance && global_iter < data.max_iterations){
00066                 #pragma omp single
00067                 {
00068                     // 1. Handle communication with other processes (Exchange Ghost Cells)
00069                     MPI_Sendrecv(u_h_local.data() + up_row * data.n, data.n, MPI_DOUBLE, rank_up, 0,
00070                                  u_h_local.data() + (total_rows - 1) * data.n, data.n, MPI_DOUBLE,
      rank_down, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
00071 
00072                     MPI_Sendrecv(u_h_local.data() + (total_rows - 1 - down_row) * data.n, data.n,
      MPI_DOUBLE, rank_down, 1,
00073                                  u_h_local.data(), data.n, MPI_DOUBLE, rank_up, 1, MPI_COMM_WORLD,
      MPI_STATUS_IGNORE);
00074 
00075                     // Save state for global error computation later
00076                     u_h_old_global_step = u_h_local;
00077                 }
00078 
00079 
00080             // Local Schwarz iterations
00081                 for(unsigned local_iter = 0; local_iter < MAX_SCHWARZ_ITERATIONS; ++local_iter) {
00082 
00083                     #pragma omp for collapse(2)
00084                         for(unsigned i = 1; i < total_rows - 1; i++){
00085                             for(unsigned j = 1; j < data.n - 1; j++){
00086                                 u_h_new_local(i, j) = 0.25 * (u_h_local(i-1, j) + u_h_local(i+1, j) +
      u_h_local(i, j-1) + u_h_local(i, j+1) + h2 * data.f0(meshX(i - up_row, j), meshY(i - up_row, j)));

00087                             }
00088                         }
00089 
00090                     #pragma omp single
00091                     {
00092                         // Preserve ghost information - only rows, as columns undergo BC assignement
      for every processor
00093                         u_h_new_local.row(0) = u_h_local.row(0);
00094                         u_h_new_local.row(total_rows - 1) = u_h_local.row(total_rows - 1);
00095                         //u_h_new_local.col(0) = u_h_local.col(0);
00096                         //u_h_new_local.col(data.n - 1) = u_h_local.col(data.n - 1);
00097 
00098                         // Apply Neumann boundary conditions locally
00099                         if constexpr(boundary_condition == BoundaryCondition::NEUMANN ||
      boundary_condition == BoundaryCondition::ROBIN)
00100 
      apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_h_new_local, data, meshX,
      meshY, u_h_local, mpi_rank, mpi_size);
00101 
00102                         u_h_local.swap(u_h_new_local);
00103                     }
00104                 }
00105                 // ================================================================
00106                 #pragma omp single
00107                 {
00108                 // 2. Apply zero-average constraint GLOBALLY at the macro-step
```

```
00109                     if constexpr(boundary_condition == BoundaryCondition::NEUMANN){
00110                         double avg = 0.0;
00111                         const double local_sum = u_h_local.block(up_row, 0, local_rows, data.n).sum();
00112                         MPI_Allreduce(&local_sum, &avg, 1, MPI_DOUBLE, MPI_SUM, MPI_COMM_WORLD);
00113                         // Shift the entire local domain to enforce uniqueness
00114                         u_h_local.array() -= avg / (data.n * data.n);
00115                     }
00116 
00117 
00118                     // 3. Global error computation based on real rows ONLY
00119                     const double macro_local_err = (u_h_local.block(up_row, 0, local_rows, data.n) -
00120                                                     u_h_old_global_step.block(up_row, 0, local_rows,
      data.n)).squaredNorm();
00121 
00122                     MPI_Allreduce(&macro_local_err, &global_err, 1, MPI_DOUBLE, MPI_SUM,
      MPI_COMM_WORLD);
00123                     global_err = std::sqrt(h * global_err);
00124 
00125                     global_iter++;
00126 
00127                 }// Single
00128             }// While
00129         }// Parallel
00130 
00131         // Prepare communication for Gather
00132         std::vector<int> recv_counts(mpi_size);
00133         std::vector<int> displs(mpi_size);
00134 
00135         for(int i = 0; i < mpi_size; i++){
00136             recv_counts[i] = data.n * (data.n / mpi_size + (static_cast<unsigned>(i) < remainder_rows
      ? 1 : 0));
00137             displs[i] = i == 0 ? 0 : displs[i-1] + recv_counts[i-1];
00138         }
00139 
00140         // Assemble global solution and mesh
00141         eigenMatrix u_h_global(data.n, data.n);
00142         eigenMatrix meshX_global(data.n, data.n);
00143         eigenMatrix meshY_global(data.n, data.n);
00144 
00145         MPI_Gatherv(u_h_local.data() + up_row * data.n, local_rows * data.n, MPI_DOUBLE,
      u_h_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00146         MPI_Gatherv(meshX.data(), local_rows * data.n, MPI_DOUBLE, meshX_global.data(),
      recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00147         MPI_Gatherv(meshY.data(), local_rows * data.n, MPI_DOUBLE, meshY_global.data(),
      recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00148 
00149         // Return results
00150         result.u_h.swap(u_h_global);
00151         result.X.swap(meshX_global);
00152         result.Y.swap(meshY_global);
00153         result.iterations = global_iter;
00154         result.iteration_residue = global_err;
00155         if(mpi_rank == 0) result.valid = true;
00156 
00157         return result;
00158     }
00159 }// namespace laplacian_solvers
00160 
00161 #endif // LAPLACIAN_SOLVERS_PARALLEL_SCHWARZ_IMPLEMENTATION_HPP
```

8.15 include/laplacian_solvers_postprocessing.hpp File Reference

Post-processing, console printing, and VTK export routines.

```
#include "laplacian_solvers.hpp"
#include <filesystem>
```

Include dependency graph for laplacian_solvers_postprocessing.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.15.1 Detailed Description

Post-processing, console printing, and VTK export routines.

8.16 laplacian_solvers_postprocessing.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_POSTPROCESSING_HPP
00002 #define LAPLACIAN_SOLVERS_POSTPROCESSING_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include <filesystem>
00006
00012 namespace laplacian_solvers{
00013
00017     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00018     void Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::print() const{
00019         print_mesh();
00020         print_solution();
00021         print_exact_solution();
00022     }
00023
00029     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00030     void Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::print_mesh()
      const{
00031
00032         eigenMatrix meshX_global(data.n, data.n), meshY_global(data.n, data.n);
00033
00034         if constexpr (execution_mode == ExecutionMode::PARALLEL) { // Gather data from other processes
00035
00036
00037             const int remainder_rows = static_cast<int>(data.n) % mpi_size;
00038             const int local_rows = static_cast<int>(data.n) / mpi_size + (mpi_rank < remainder_rows ?
      1 : 0);
00039
00040             std::vector<int> recv_counts(mpi_size);
00041             std::vector<int> displs(mpi_size);
00042
00043             for(int i = 0; i < mpi_size; i++){
00044                 recv_counts[i] = static_cast<int>(data.n) * (static_cast<int>(data.n) / mpi_size + (i
      < remainder_rows ? 1 : 0));
00045                 displs[i] = (i == 0) ? 0 : displs[i-1] + recv_counts[i-1];
00046             }
00047
00048             MPI_Gatherv(meshX.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
      meshX_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00049             MPI_Gatherv(meshY.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
      meshY_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00050
00051
00052         } else{
00053             meshX_global = meshX;
00054             meshY_global = meshY;
00055         }
00056
00057         if(mpi_rank != 0) return; // Only the root process
00058         std::cout « "Mesh points (x, y):" « std::endl;
00059
00060         for(unsigned i = 0; i < data.n; i++){
00061             for(unsigned j = 0; j < data.n; j++) std::cout « "(" « meshX_global(i, j) « ", " «
      meshY_global(i, j) « ") ";
00062              std::cout « std::endl;
00063         }
00064
00065     }
00066
00072     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00073     void Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::print_solution()
      const{
00074
00075         int mpi_rank;
00076         MPI_Comm_rank(MPI_COMM_WORLD, &mpi_rank);
00077
00078         if(mpi_rank != 0) return;
00079
00080         std::cout « "Discrete solution u_h:" « std::endl;
```

```
00081
00082         if(result.iteration_residue == -1.) {
00083           std::cout « "Solver was not called" « std::endl;
00084           return;
00085         }
00086
00087         for(unsigned i = 0; i < data.n; i++){
00088             for(unsigned j = 0; j < data.n; j++) std::cout « result.u_h(i, j) « " ";
00089             std::cout « std::endl;
00090         }
00091
00092         std::cout « "Residue error: " « result.iteration_residue « std::endl;
00093         std::cout « "Total iterations: " « result.iterations « std::endl;
00094     }
00095
00096
00100     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00101     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::print_exact_solution()
      const{
00102
00103         eigenMatrix exact_solution(data.n, data.n);
00104
00105         if constexpr (execution_mode == ExecutionMode::PARALLEL) { // Gather data from other processes
00106
00107             const int remainder_rows = static_cast<int>(data.n) % mpi_size;
00108             const int local_rows = static_cast<int>(data.n) / mpi_size + (mpi_rank < remainder_rows ?
      1 : 0);
00109
00110             std::vector<int> recv_counts(mpi_size);
00111             std::vector<int> displs(mpi_size);
00112
00113             for(int i = 0; i < mpi_size; i++){
00114                 recv_counts[i] = static_cast<int>(data.n) * (static_cast<int>(data.n) / mpi_size + (i
      < remainder_rows ? 1 : 0));
00115                 displs[i] = (i == 0) ? 0 : displs[i-1] + recv_counts[i-1];
00116             }
00117
00118             MPI_Gatherv(u_exact.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
      exact_solution.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00119
00120         } else{
00121             exact_solution = u_exact;
00122         }
00123
00124         if(mpi_rank != 0) return; // Only the root process
00125
00126         std::cout « "Exact solution u_exact:" « std::endl;
00127
00128         for(unsigned i = 0; i < data.n; i++){
00129             for(unsigned j = 0; j < data.n; j++) std::cout « exact_solution(i, j) « " ";
00130             std::cout « std::endl;
00131         }
00132     }
00133
00141     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00142     void
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::export_to_vtk(const
      std::string& filename) const {
00143
00144         // 1. Global matrices to gather data from all processes
00145         eigenMatrix meshX_global(data.n, data.n), meshY_global(data.n, data.n);
00146         eigenMatrix exact_solution_global(data.n, data.n);
00147
00148
00149         // 2. Gather data from all processes if in parallel mode, otherwise just copy local to global
      matrices
00150         if constexpr (execution_mode == ExecutionMode::PARALLEL) {
00151             //int mpi_size;
00152             //MPI_Comm_size(MPI_COMM_WORLD, &mpi_size);
00153
00154             const int remainder_rows = static_cast<int>(data.n) % mpi_size;
00155             const int local_rows = static_cast<int>(data.n) / mpi_size + (mpi_rank < remainder_rows ?
      1 : 0);
00156
00157             std::vector<int> recv_counts(mpi_size);
00158             std::vector<int> displs(mpi_size);
00159
00160             for(int i = 0; i < mpi_size; i++){
00161                 recv_counts[i] = static_cast<int>(data.n) * (static_cast<int>(data.n) / mpi_size + (i
      < remainder_rows ? 1 : 0));
00162                 displs[i] = (i == 0) ? 0 : displs[i-1] + recv_counts[i-1];
00163             }
00164
00165             MPI_Gatherv(meshX.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
```

```
    meshX_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00166             MPI_Gatherv(meshY.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
    meshY_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00167             MPI_Gatherv(u_exact.data(), local_rows * static_cast<int>(data.n), MPI_DOUBLE,
    exact_solution_global.data(), recv_counts.data(), displs.data(), MPI_DOUBLE, 0, MPI_COMM_WORLD);
00168
00169         } else {
00170             // Caso sequenziale: le matrici globali copiano direttamente quelle locali
00171             meshX_global = meshX;
00172             meshY_global = meshY;
00173             exact_solution_global = u_exact;
00174         }
00175
00176         // 3. Root process handles file writing, while others return immediately
00177         if (mpi_rank != 0) {
00178             return;
00179         }
00180
00181         // 4. Ensure output directory exists and handle filename conflicts
00182         std::filesystem::path output_dir = "vtk_files_folder";
00183         std::filesystem::create_directories(output_dir);
00184         std::filesystem::path full_path = output_dir / filename;
00185
00186         if (std::filesystem::exists(full_path)) {
00187             std::string stem = full_path.stem().string();
00188             std::string extension = full_path.extension().string();
00189
00190             unsigned counter = 1;
00191             while (std::filesystem::exists(full_path)) {
00192                 std::string new_filename = stem + "_" + std::to_string(counter) + extension;
00193                 full_path = output_dir / new_filename;
00194                 counter++;
00195             }
00196         }
00197
00198         std::ofstream vtk_file(full_path);
00199         if(!vtk_file.is_open()) {
00200             throw std::runtime_error("Could not open / create file for writing: " +
    full_path.string());
00201         }
00202
00203         // 5. VTK Standard Header
00204         vtk_file « "# vtk DataFile Version 3.0\n";
00205         vtk_file « "Laplacian Solver Output\n";
00206         vtk_file « "ASCII\n";
00207         vtk_file « "DATASET STRUCTURED_GRID\n";
00208
00209         // 6. Grid Topology Definition
00210         vtk_file « "DIMENSIONS " « data.n « " " « data.n « " 1\n";
00211
00212         size_t total_points = data.n * data.n;
00213         vtk_file « "POINTS " « total_points « " double\n";
00214
00215         for (unsigned j = 0; j < data.n; ++j) {
00216             for (unsigned i = 0; i < data.n; ++i) {
00217                 vtk_file « meshX_global(j, i) « " " « meshY_global(j, i) « " 0.0\n";
00218             }
00219         }
00220
00221         // 7. Point-Data Section
00222         vtk_file « "POINT_DATA " « total_points « "\n";
00223
00224         // Field 1: Computed Numerical Solution (u_h)
00225         vtk_file « "SCALARS Numerical_Solution float 1\n";
00226         vtk_file « "LOOKUP_TABLE default\n";
00227         for (unsigned j = 0; j < data.n; ++j) {
00228             for (unsigned i = 0; i < data.n; ++i) {
00229                 vtk_file « result.u_h(j, i) « "\n";
00230             }
00231         }
00232
00233         // Field 2: Analytical / Exact Solution (u_exact)
00234         vtk_file « "SCALARS Exact_Solution float 1\n";
00235         vtk_file « "LOOKUP_TABLE default\n";
00236         for (unsigned j = 0; j < data.n; ++j) {
00237             for (unsigned i = 0; i < data.n; ++i) {
00238                 vtk_file « exact_solution_global(j, i) « "\n";
00239             }
00240         }
00241
00242         vtk_file.close();
00243     }
00244
00245 } // namespace laplacian_solvers
00246
00247 #endif // LAPLACIAN_SOLVERS_POSTPROCESSING_HPP
```

8.17 include/laplacian_solvers_sequential_implementations.hpp File Reference

Implementation of the sequential Jacobi iterative solver.

```
#include "laplacian_solvers.hpp"
#include "laplacian_solvers_boundary_conditions.hpp"
#include <iostream>
#include <cmath>
#include <vector>
#include <fstream>
```

Include dependency graph for laplacian_solvers_sequential_implementations.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

8.17.1 Detailed Description

Implementation of the sequential Jacobi iterative solver.

8.18 laplacian_solvers_sequential_implementations.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_SEQUENTIAL_IMPLEMENTATIONS_HPP
00002 #define LAPLACIAN_SOLVERS_SEQUENTIAL_IMPLEMENTATIONS_HPP
00003
00004 #include "laplacian_solvers.hpp"
00005 #include "laplacian_solvers_boundary_conditions.hpp"
00006
00007 #include <iostream>
00008 #include <cmath>
00009 #include <vector>
00010 #include <fstream>
00011
00017 namespace laplacian_solvers{
00018
00030     template <SolverType solver_type, BoundaryCondition boundary_condition, ExecutionMode
      execution_mode, typename funcType>
00031     Result_Struct
      Laplacian_Solver<solver_type, boundary_condition, execution_mode, funcType>::jacobi_sequential(){
00032
00033         // Initialize the solver
00034         eigenMatrix u_h = eigenMatrix::Zero(data.n, data.n);
00035
00036         // Dirichlet and Robin BCs must be one the initial guess
00037         if constexpr(boundary_condition == BoundaryCondition::DIRICHLET || boundary_condition ==
      BoundaryCondition::ROBIN)
00038             apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_h, data, meshX,
      meshY, u_h);
00039
00040         eigenMatrix u_new(u_h);
00041         unsigned iter = 0;
00042         const double tol_squared = data.tolerance * data.tolerance;
00043         double err = tol_squared + 1.0;
00044         const double h2 = h * h;
00045         const unsigned last = data.n - 1;
00046
00047
00048         // Main iteration loop
00049         while (err > tol_squared && iter < data.max_iterations) {
00050
```

```
00051             // Compute next iteration
00052             for (unsigned i = 1; i < last; ++i) {
00053                 for (unsigned j = 1; j < last; ++j) {
00054                     u_new(i, j) = 0.25 * (u_h(i-1, j) + u_h(i+1, j) + u_h(i, j-1) + u_h(i, j+1) + h2 *
      data.f0(meshX(i, j), meshY(i, j)));
00055                 }
00056             }
00057
00058             // Neumann and Robin BCs must be applied at each iteration
00059             if constexpr(boundary_condition == BoundaryCondition::NEUMANN || boundary_condition ==
      BoundaryCondition::ROBIN)
00060                 apply_boundary_condition<boundary_condition, execution_mode, funcType>(u_new, data,
      meshX, meshY, u_h);
00061
00062             // Neumann requires normalization to ensure unicity of the solution
00063             if constexpr(boundary_condition == BoundaryCondition::NEUMANN){
00064                 const double mean = u_h.sum() / (double)(data.n * data.n);
00065                 u_h.array() -= mean;
00066             }
00067
00068             // Compute error using modified l2 norm and prepare for next iteration
00069             err = h * (u_new - u_h).squaredNorm();
00070             u_h = u_new;
00071             iter++;
00072         }
00073
00074         // Update result struct with final solution and iteration info
00075         result.u_h = u_h;
00076         result.iterations = iter;
00077         result.X = meshX;
00078         result.Y = meshY;
00079         result.iteration_residue = std::sqrt(err);
00080         result.valid = true;
00081         return result;
00082     }
00083 } // namespace laplacian_solvers
00084
00085 #endif // LAPLACIAN_SOLVERS_SEQUENTIAL_IMPLEMENTATIONS_HPP
```

8.19 include/laplacian_solvers_test_1.hpp File Reference

Integration test suite covering sequential, parallel, and convergence benchmarks.

```
#include <iostream>
#include <cmath>
#include <functional>
#include <mpi.h>
#include "laplacian_solvers.hpp"
#include "laplacian_convergence_test.hpp"
#include "laplacian_solvers_postprocessing.hpp"
```

Include dependency graph for laplacian_solvers_test_1.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

Typedefs

- using laplacian_solvers::funcType = std::function< double(double, double)>

  Type alias for standard 2D spatial functions.

Functions

- Data_Struct< funcType > laplacian_solvers::create_default_data ()

  Generates a baseline configuration structure with default parameters.
- void laplacian_solvers::run_test_1_dirichlet_sequential (int mpi_rank)

  Test 1: Evaluates a sequential Jacobi solver under homogeneous Dirichlet boundaries.
- void laplacian_solvers::run_test_2_neumann_sequential (int mpi_rank)

  Test 2: Evaluates a sequential Jacobi solver under pure Neumann boundaries.
- void laplacian_solvers::run_test_3_robin_sequential (int mpi_rank)

  Test 3: Evaluates a sequential Jacobi solver under Robin boundary conditions.
- void laplacian_solvers::run_test_4_dirichlet_parallel (int mpi_rank)

  Test 4: Evaluates a parallel MPI Jacobi solver under Dirichlet boundaries.
- void laplacian_solvers::run_test_5_neumann_parallel (int mpi_rank)

  Test 5: Evaluates a parallel MPI Jacobi solver under Neumann boundaries.
- void laplacian_solvers::run_test_6_robin_parallel (int mpi_rank)

  Test 6: Evaluates a parallel MPI Jacobi solver under Robin boundaries.
- void laplacian_solvers::run_test_7_dirichlet_parallel_schwarz (int mpi_rank)

  Test 7: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Dirichlet boundaries.
- void laplacian_solvers::run_test_8_neumann_parallel_schwarz (int mpi_rank)

  Test 8: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Neumann boundaries.
- void laplacian_solvers::run_test_9_robin_parallel_schwarz (int mpi_rank)

  Test 9: Evaluates a hybrid MPI+OpenMP Schwarz domain decomposition solver under Robin boundaries.
- void laplacian_solvers::run_test_10_dirichlet_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 10: Performs error convergence analysis and speedup profiling for Dirichlet Jacobi routines.
- void laplacian_solvers::run_test_11_neumann_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 11: Performs error convergence analysis and speedup profiling for Neumann Jacobi routines.
- void laplacian_solvers::run_test_12_robin_sequential_vs_parallel_jacobi (int mpi_rank)

  Test 12: Performs error convergence analysis and speedup profiling for Robin Jacobi routines.
- void laplacian_solvers::run_test_13_dirichlet_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 13: Performs error convergence analysis and speedup profiling for Dirichlet Schwarz domain decomposition.
- void laplacian_solvers::run_test_14_neumann_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 14: Performs error convergence analysis and speedup profiling for Neumann Schwarz domain decomposition.
- void laplacian_solvers::run_test_15_robin_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 15: Performs error convergence analysis and speedup profiling for Robin Schwarz domain decomposition.

8.19.1 Detailed Description

Integration test suite covering sequential, parallel, and convergence benchmarks.

8.20 laplacian_solvers_test_1.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_TEST_1_HPP
00002 #define LAPLACIAN_SOLVERS_TEST_1_HPP
00003
00004 #include <iostream>
00005 #include <cmath>
00006 #include <functional>
00007 #include <mpi.h>
00008 #include "laplacian_solvers.hpp"
00009 #include "laplacian_convergence_test.hpp"
00010 #include "laplacian_solvers_postprocessing.hpp"
00011
```

```
00017 namespace laplacian_solvers {
00018
00020     using funcType = std::function<double(double, double)>;
00021
00026     inline Data_Struct<funcType> create_default_data() {
00027         Data_Struct<funcType> data;
00028         data.n = 8;
00029         data.x1 = 0.0;
00030         data.x2 = 1.0;
00031         data.tolerance = 1e-6;
00032         data.max_iterations = 100000;
00033         data.alpha = 1.0;
00034         return data;
00035     }
00036
00037     // ================================================================================
00038     // TEST 1: DIRICHLET SEQUENTIAL (Jacobi)
00039     // ================================================================================
00044     inline void run_test_1_dirichlet_sequential(int mpi_rank) {
00045         if (mpi_rank == 0) {
00046             std::cout « "\n=== Running Test 1: DIRICHLET SEQUENTIAL (Jacobi) ===" « std::endl;
00047         }
00048
00049         auto data = create_default_data();
00050         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::sin(M_PI * x) *
      std::sin(M_PI * y); };
00051         data.f1 = [](double x, double y) { return 0.0; };
00052         data.f2 = [](double x, double y) { return 0.0; };
00053         data.f3 = [](double x, double y) { return 0.0; };
00054         data.f4 = [](double x, double y) { return 0.0; };
00055         data.u_exact_lambda = [](double x, double y) { return std::sin(M_PI * x) * std::sin(M_PI * y);
      };
00056
00057
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::DIRICHLET, ExecutionMode::SEQUENTIAL, funcTyp
      solver(data);
00058         solver.build_mesh();
00059         solver.solve();
00060         solver.print();
00061         solver.export_to_vtk("test_1_dirichlet_sequential.vtk");
00062     }
00063
00064     // ================================================================================
00065     // TEST 2: NEUMANN SEQUENTIAL (Jacobi)
00066     // ================================================================================
00071     inline void run_test_2_neumann_sequential(int mpi_rank) {
00072         if (mpi_rank == 0) {
00073             std::cout « "\n=== Running Test 2: NEUMANN SEQUENTIAL (Jacobi) ===" « std::endl;
00074         }
00075
00076         auto data = create_default_data();
00077         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::cos(M_PI * x) *
      std::cos(M_PI * y); };
00078         data.f1 = [](double x, double y) { return 0.0; };
00079         data.f2 = [](double x, double y) { return 0.0; };
00080         data.f3 = [](double x, double y) { return 0.0; };
00081         data.f4 = [](double x, double y) { return 0.0; };
00082         data.u_exact_lambda = [](double x, double y) { return std::cos(M_PI * x) * std::cos(M_PI * y);
      };
00083
00084
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::NEUMANN, ExecutionMode::SEQUENTIAL, funcType>
      solver(data);
00085         solver.build_mesh();
00086         solver.solve();
00087         solver.print();
00088         solver.export_to_vtk("test_2_neumann_sequential.vtk");
00089     }
00094     // ================================================================================
00095     // TEST 3: ROBIN SEQUENTIAL (Jacobi)
00096     // ================================================================================
00097     inline void run_test_3_robin_sequential(int mpi_rank) {
00098         if (mpi_rank == 0) {
00099             std::cout « "\n=== Running Test 3: ROBIN SEQUENTIAL (Jacobi) ===" « std::endl;
00100         }
00101
00102         auto data = create_default_data();
00103         data.f0 = [](double x, double y) { return -2.0 * std::exp(x + y); };
00104         data.f1 = [](double x, double y) { return 0.0; };
00105         data.f2 = [](double x, double y) { return 2.0 * std::exp(1.0 + y); };
00106         data.f3 = [](double x, double y) { return 2.0 * std::exp(x + 1.0); };
00107         data.f4 = [](double x, double y) { return 0.0; };
00108         data.u_exact_lambda = [](double x, double y) { return std::exp(x + y); };
00109
00110
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::ROBIN, ExecutionMode::SEQUENTIAL, funcType>
      solver(data);
```

```
00111         solver.build_mesh();
00112         solver.solve();
00113         solver.print();
00114         solver.export_to_vtk("test_3_robin_sequential.vtk");
00115     }
00116
00117     // ======================================================================
00118     // TEST 4: DIRICHLET PARALLEL (Jacobi)
00119     // ======================================================================
00124     inline void run_test_4_dirichlet_parallel(int mpi_rank) {
00125         if (mpi_rank == 0) {
00126             std::cout « "\n=== Running Test 4: DIRICHLET PARALLEL (Jacobi) ===" « std::endl;
00127         }
00128
00129         auto data = create_default_data();
00130         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::sin(M_PI * x) *
      std::sin(M_PI * y); };
00131         data.f1 = [](double x, double y) { return 0.0; };
00132         data.f2 = [](double x, double y) { return 0.0; };
00133         data.f3 = [](double x, double y) { return 0.0; };
00134         data.f4 = [](double x, double y) { return 0.0; };
00135         data.u_exact_lambda = [](double x, double y) { return std::sin(M_PI * x) * std::sin(M_PI * y);
      };
00136
00137
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::DIRICHLET, ExecutionMode::PARALLEL, funcType>
      solver(data);
00138         solver.build_mesh();
00139         solver.solve();
00140         solver.print();
00141         solver.export_to_vtk("test_4_dirichlet_parallel_jacobi.vtk");
00142     }
00143
00144     // ======================================================================
00145     // TEST 5: NEUMANN PARALLEL (Jacobi)
00146     // ======================================================================
00151     inline void run_test_5_neumann_parallel(int mpi_rank) {
00152         if (mpi_rank == 0) {
00153             std::cout « "\n=== Running Test 5: NEUMANN PARALLEL (Jacobi) ===" « std::endl;
00154         }
00155
00156         auto data = create_default_data();
00157         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::cos(M_PI * x) *
      std::cos(M_PI * y); };
00158         data.f1 = [](double x, double y) { return 0.0; };
00159         data.f2 = [](double x, double y) { return 0.0; };
00160         data.f3 = [](double x, double y) { return 0.0; };
00161         data.f4 = [](double x, double y) { return 0.0; };
00162         data.u_exact_lambda = [](double x, double y) { return std::cos(M_PI * x) * std::cos(M_PI * y);
      };
00163
00164
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::NEUMANN, ExecutionMode::PARALLEL, funcType>
      solver(data);
00165         solver.build_mesh();
00166         solver.solve();
00167         solver.print();
00168         solver.export_to_vtk("test_5_neumann_parallel_jacobi.vtk");
00169     }
00170
00171     // ======================================================================
00172     // TEST 6: ROBIN PARALLEL (Jacobi)
00173     // ======================================================================
00178     inline void run_test_6_robin_parallel(int mpi_rank) {
00179         if (mpi_rank == 0) {
00180             std::cout « "\n=== Running Test 6: ROBIN PARALLEL (Jacobi) ===" « std::endl;
00181         }
00182
00183         auto data = create_default_data();
00184         data.f0 = [](double x, double y) { return -2.0 * std::exp(x + y); };
00185         data.f1 = [](double x, double y) { return 0.0; };
00186         data.f2 = [](double x, double y) { return 2.0 * std::exp(1.0 + y); };
00187         data.f3 = [](double x, double y) { return 2.0 * std::exp(x + 1.0); };
00188         data.f4 = [](double x, double y) { return 0.0; };
00189         data.u_exact_lambda = [](double x, double y) { return std::exp(x + y); };
00190
00191
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::ROBIN, ExecutionMode::PARALLEL, funcType>
      solver(data);
00192         solver.build_mesh();
00193         solver.solve();
00194         solver.print();
00195         solver.export_to_vtk("test_6_robin_parallel_jacobi.vtk");
00196     }
00197
00198     // ======================================================================
00199     // TEST 7: DIRICHLET PARALLEL (Schwarz)
```

```
00200     // ========================================================================
00205     inline void run_test_7_dirichlet_parallel_schwarz(int mpi_rank) {
00206         if (mpi_rank == 0) {
00207             std::cout « "\n=== Running Test 7: DIRICHLET PARALLEL (Schwarz) ===" « std::endl;
00208         }
00209 
00210         auto data = create_default_data();
00211         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::sin(M_PI * x) *
      std::sin(M_PI * y); };
00212         data.f1 = [](double x, double y) { return 0.0; };
00213         data.f2 = [](double x, double y) { return 0.0; };
00214         data.f3 = [](double x, double y) { return 0.0; };
00215         data.f4 = [](double x, double y) { return 0.0; };
00216         data.u_exact_lambda = [](double x, double y) { return std::sin(M_PI * x) * std::sin(M_PI * y);
      };
00217 
00218 
      laplacian_solvers::Laplacian_Solver<SolverType::SCHWARZ, BoundaryCondition::DIRICHLET, ExecutionMode::PARALLEL, funcType
      solver(data);
00219         solver.build_mesh();
00220         solver.solve();
00221         solver.print();
00222         solver.export_to_vtk("test_7_dirichlet_parallel_schwarz.vtk");
00223     }
00224 
00225     // ========================================================================
00226     // TEST 8: NEUMANN PARALLEL (Schwarz)
00227     // ========================================================================
00232     inline void run_test_8_neumann_parallel_schwarz(int mpi_rank) {
00233         if (mpi_rank == 0) {
00234             std::cout « "\n=== Running Test 8: NEUMANN PARALLEL (Schwarz) ===" « std::endl;
00235         }
00236 
00237         auto data = create_default_data();
00238         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::cos(M_PI * x) *
      std::cos(M_PI * y); };
00239         data.f1 = [](double x, double y) { return 0.0; };
00240         data.f2 = [](double x, double y) { return 0.0; };
00241         data.f3 = [](double x, double y) { return 0.0; };
00242         data.f4 = [](double x, double y) { return 0.0; };
00243         data.u_exact_lambda = [](double x, double y) { return std::cos(M_PI * x) * std::cos(M_PI * y);
      };
00244 
00245 
      laplacian_solvers::Laplacian_Solver<SolverType::SCHWARZ, BoundaryCondition::NEUMANN, ExecutionMode::PARALLEL, funcType>
      solver(data);
00246         solver.build_mesh();
00247         solver.solve();
00248         solver.print();
00249         solver.export_to_vtk("test_8_neumann_parallel_schwarz.vtk");
00250     }
00251 
00252     // ========================================================================
00253     // TEST 9: ROBIN PARALLEL (Schwarz)
00254     // ========================================================================
00259     inline void run_test_9_robin_parallel_schwarz(int mpi_rank) {
00260         if (mpi_rank == 0) {
00261             std::cout « "\n=== Running Test 9: ROBIN PARALLEL (Schwarz) ===" « std::endl;
00262         }
00263 
00264         auto data = create_default_data();
00265         data.f0 = [](double x, double y) { return -2.0 * std::exp(x + y); };
00266         data.f1 = [](double x, double y) { return 0.0; };
00267         data.f2 = [](double x, double y) { return 2.0 * std::exp(1.0 + y); };
00268         data.f3 = [](double x, double y) { return 2.0 * std::exp(x + 1.0); };
00269         data.f4 = [](double x, double y) { return 0.0; };
00270         data.u_exact_lambda = [](double x, double y) { return std::exp(x + y); };
00271 
00272 
      laplacian_solvers::Laplacian_Solver<SolverType::SCHWARZ, BoundaryCondition::ROBIN, ExecutionMode::PARALLEL, funcType>
      solver(data);
00273         solver.build_mesh();
00274         solver.solve();
00275         solver.print();
00276         solver.export_to_vtk("test_9_robin_parallel_schwarz.vtk");
00277     }
00278 
00279     // ========================================================================
00280     // TEST 10: DIRICHLET SEQUENTIAL vs PARALLEL (Jacobi)
00281     // ========================================================================
00286     inline void run_test_10_dirichlet_sequential_vs_parallel_jacobi(int mpi_rank) {
00287         if (mpi_rank == 0) {
00288             std::cout « "\n=== Running Test 10: DIRICHLET SEQUENTIAL vs PARALLEL (Jacobi) ===" «
      std::endl;
00289         }
00290 
00291         auto data = create_default_data();
```

```
00292         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::sin(M_PI * x) *
      std::sin(M_PI * y); };
00293         data.f1 = [](double x, double y) { return 0.0; };
00294         data.f2 = [](double x, double y) { return 0.0; };
00295         data.f3 = [](double x, double y) { return 0.0; };
00296         data.f4 = [](double x, double y) { return 0.0; };
00297         data.u_exact_lambda = [](double x, double y) { return std::sin(M_PI * x) * std::sin(M_PI * y);
      };
00298
00299
      laplacian_solvers::Convergence_Test<SolverType::JACOBI, BoundaryCondition::DIRICHLET, funcType>
      convergence_test(data);
00300         auto convergence_results = convergence_test.run_convergence_test();
00301         convergence_test.print_convergence_results();
00302     }
00303
00304     // ========================================================================
00305     // TEST 11: NEUMANN SEQUENTIAL vs PARALLEL (Jacobi)
00306     // ========================================================================
00311     inline void run_test_11_neumann_sequential_vs_parallel_jacobi(int mpi_rank) {
00312         if (mpi_rank == 0) {
00313             std::cout « "\n=== Running Test 11: NEUMANN SEQUENTIAL vs PARALLEL (Jacobi) ===" «
      std::endl;
00314         }
00315
00316         auto data = create_default_data();
00317         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::cos(M_PI * x) *
      std::cos(M_PI * y); };
00318         data.f1 = [](double x, double y) { return 0.0; };
00319         data.f2 = [](double x, double y) { return 0.0; };
00320         data.f3 = [](double x, double y) { return 0.0; };
00321         data.f4 = [](double x, double y) { return 0.0; };
00322         data.u_exact_lambda = [](double x, double y) { return std::cos(M_PI * x) * std::cos(M_PI * y);
      };
00323
00324         laplacian_solvers::Convergence_Test<SolverType::JACOBI, BoundaryCondition::NEUMANN, funcType>
      convergence_test(data);
00325         auto convergence_results = convergence_test.run_convergence_test();
00326         convergence_test.print_convergence_results();
00327     }
00328
00329     // ========================================================================
00330     // TEST 12: ROBIN SEQUENTIAL vs PARALLEL (Jacobi)
00331     // ========================================================================
00336     inline void run_test_12_robin_sequential_vs_parallel_jacobi(int mpi_rank) {
00337         if (mpi_rank == 0) {
00338             std::cout « "\n=== Running Test 12: ROBIN SEQUENTIAL vs PARALLEL (Jacobi) ===" «
      std::endl;
00339         }
00340
00341         auto data = create_default_data();
00342         data.f0 = [](double x, double y) { return -2.0 * std::exp(x + y); };
00343         data.f1 = [](double x, double y) { return 0.0; };
00344         data.f2 = [](double x, double y) { return 2.0 * std::exp(1.0 + y); };
00345         data.f3 = [](double x, double y) { return 2.0 * std::exp(x + 1.0); };
00346         data.f4 = [](double x, double y) { return 0.0; };
00347         data.u_exact_lambda = [](double x, double y) { return std::exp(x + y); };
00348
00349         laplacian_solvers::Convergence_Test<SolverType::JACOBI, BoundaryCondition::ROBIN, funcType>
      convergence_test(data);
00350         auto convergence_results = convergence_test.run_convergence_test();
00351         convergence_test.print_convergence_results();
00352     }
00353
00354     // ========================================================================
00355     // TEST 13: DIRICHLET SEQUENTIAL vs PARALLEL (Schwarz)
00356     // ========================================================================
00361     inline void run_test_13_dirichlet_sequential_vs_parallel_schwarz(int mpi_rank) {
00362         if (mpi_rank == 0) {
00363             std::cout « "\n=== Running Test 13: DIRICHLET SEQUENTIAL vs PARALLEL (Schwarz) ===" «
      std::endl;
00364         }
00365
00366         auto data = create_default_data();
00367         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::sin(M_PI * x) *
      std::sin(M_PI * y); };
00368         data.f1 = [](double x, double y) { return 0.0; };
00369         data.f2 = [](double x, double y) { return 0.0; };
00370         data.f3 = [](double x, double y) { return 0.0; };
00371         data.f4 = [](double x, double y) { return 0.0; };
00372         data.u_exact_lambda = [](double x, double y) { return std::sin(M_PI * x) * std::sin(M_PI * y);
      };
00373
00374
      laplacian_solvers::Convergence_Test<SolverType::SCHWARZ, BoundaryCondition::DIRICHLET, funcType>
      convergence_test(data);
00375         auto convergence_results = convergence_test.run_convergence_test();
```

```
00376         convergence_test.print_convergence_results();
00377     }
00378
00379     // ================================================================================
00380     // TEST 14: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz)
00381     // ================================================================================
00386     inline void run_test_14_neumann_sequential_vs_parallel_schwarz(int mpi_rank) {
00387         if (mpi_rank == 0) {
00388             std::cout << "\n=== Running Test 14: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz) ===" <<
      std::endl;
00389         }
00390
00391         auto data = create_default_data();
00392         data.f0 = [](double x, double y) { return 2.0 * M_PI * M_PI * std::cos(M_PI * x) *
      std::cos(M_PI * y); };
00393         data.f1 = [](double x, double y) { return 0.0; };
00394         data.f2 = [](double x, double y) { return 0.0; };
00395         data.f3 = [](double x, double y) { return 0.0; };
00396         data.f4 = [](double x, double y) { return 0.0; };
00397         data.u_exact_lambda = [](double x, double y) { return std::cos(M_PI * x) * std::cos(M_PI * y);
      };
00398
00399         laplacian_solvers::Convergence_Test<SolverType::SCHWARZ, BoundaryCondition::NEUMANN, funcType>
      convergence_test(data);
00400         auto convergence_results = convergence_test.run_convergence_test();
00401         convergence_test.print_convergence_results();
00402     }
00403
00404     // ================================================================================
00405     // TEST 15: ROBIN SEQUENTIAL vs PARALLEL (Schwarz)
00406     // ================================================================================
00411     inline void run_test_15_robin_sequential_vs_parallel_schwarz(int mpi_rank) {
00412         if (mpi_rank == 0) {
00413             std::cout << "\n=== Running Test 15: ROBIN SEQUENTIAL vs PARALLEL (Schwarz) ===" <<
      std::endl;
00414         }
00415
00416         auto data = create_default_data();
00417         data.f0 = [](double x, double y) { return -2.0 * std::exp(x + y); };
00418         data.f1 = [](double x, double y) { return 0.0; };
00419         data.f2 = [](double x, double y) { return 2.0 * std::exp(1.0 + y); };
00420         data.f3 = [](double x, double y) { return 2.0 * std::exp(x + 1.0); };
00421         data.f4 = [](double x, double y) { return 0.0; };
00422         data.u_exact_lambda = [](double x, double y) { return std::exp(x + y); };
00423
00424         laplacian_solvers::Convergence_Test<SolverType::SCHWARZ, BoundaryCondition::ROBIN, funcType>
      convergence_test(data);
00425         auto convergence_results = convergence_test.run_convergence_test();
00426         convergence_test.print_convergence_results();
00427     }
00428
00429 }
00430
00431 #endif // TESTS_HPP
```

8.21 include/laplacian_solvers_test_2.hpp File Reference

Extended integration test suite evaluating non-homogeneous boundary test cases.

`#include "laplacian_solvers_test_1.hpp"`
Include dependency graph for laplacian_solvers_test_2.hpp: This graph shows which files directly or indirectly include this file:

Namespaces

- namespace laplacian_solvers

  Namespace dedicated to the computation and numerical resolution of the Laplacian operator.

Functions

- void laplacian_solvers::run_test_16_dirichlet_sequential (int mpi_rank)

  Test 16: Sequential Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.
- void laplacian_solvers::run_test_17_neumann_sequential (int mpi_rank)

  Test 17: Sequential Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.
- void laplacian_solvers::run_test_18_robin_sequential (int mpi_rank)

  Test 18: Sequential Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.
- void laplacian_solvers::run_test_19_dirichlet_parallel (int mpi_rank)

  Test 19: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Dirichlet boundaries.
- void laplacian_solvers::run_test_20_neumann_parallel (int mpi_rank)

  Test 20: Parallel MPI-distributed Jacobi solver with a non-homogeneous polynomial solution under Neumann boundaries.
- void laplacian_solvers::run_test_21_robin_parallel (int mpi_rank)

  Test 21: Parallel MPI-distributed Jacobi solver with a non-homogeneous trigonometric solution under Robin boundaries.
- void laplacian_solvers::run_test_22_neumann_sequential_vs_parallel_schwarz (int mpi_rank)

  Test 22: Performance convergence benchmark for a polynomial solution under Neumann conditions using the Schwarz solver.

8.21.1 Detailed Description

Extended integration test suite evaluating non-homogeneous boundary test cases.

- Contains advanced validation tests using multi-dimensional polynomial and trigonometric exact solutions to verify convergence under complex boundary configurations.

8.22 laplacian_solvers_test_2.hpp

Go to the documentation of this file.

```
00001 #ifndef LAPLACIAN_SOLVERS_TEST_2_HPP
00002 #define LAPLACIAN_SOLVERS_TEST_2_HPP
00003
00004 #include "laplacian_solvers_test_1.hpp"
00005
00013 namespace laplacian_solvers {
00014
00015     // ========================================================================
00016     // TEST 16: DIRICHLET SEQUENTIAL (Jacobi) - Non-homogeneous Polynomial
00017     // ========================================================================
00023     inline void run_test_16_dirichlet_sequential(int mpi_rank) {
00024         if (mpi_rank == 0) {
00025             std::cout « "\n=== Running Test 16: DIRICHLET SEQUENTIAL (Jacobi) ===" « std::endl;
00026         }
00027
00028         auto data = create_default_data();
00029
00030         // Exact solution: u(x,y) = x^2*y + x*y^2 + x + y + 1
00031         data.u_exact_lambda = [](double x, double y) { return x*x*y + x*y*y + x + y + 1.0; };
00032
00033         // Forcing term: f0 = -Laplacian(u) = -2(x+y)
00034         data.f0 = [](double x, double y) { return -2.0 * (x + y); };
00035
00036         // Dirichlet boundaries: u evaluated exactly at the edges
00037         data.f1 = [](double x, double y) { return x + 1.0; };                          // Bottom
      (y=0)
00038         data.f2 = [](double x, double y) { return y*y + 2.0*y + 2.0; };                // Right
      (x=1)
00039         data.f3 = [](double x, double y) { return x*x + 2.0*x + 2.0; };                // Top
      (y=1)
```

```
00040         data.f4 = [](double x, double y) { return y + 1.0; };                              // Left
      (x=0)
00041
00042
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::DIRICHLET, ExecutionMode::SEQUENTIAL, funcTyp
      solver(data);
00043         solver.build_mesh();
00044         solver.solve();
00045         solver.print();
00046         solver.export_to_vtk("test_16_dirichlet_sequential.vtk");
00047     }
00048
00049     // ======================================================================
00050     // TEST 17: NEUMANN SEQUENTIAL (Jacobi) - Non-homogeneous Polynomial
00051     // ======================================================================
00057     inline void run_test_17_neumann_sequential(int mpi_rank) {
00058         if (mpi_rank == 0) {
00059             std::cout « "\n=== Running Test 17: NEUMANN SEQUENTIAL (Jacobi) ===" « std::endl;
00060         }
00061
00062         auto data = create_default_data();
00063
00064         // Exact solution: u(x,y) = x^2*y + x*y^2 + x + y + 1
00065         data.u_exact_lambda = [](double x, double y) { return x*x*y + x*y*y + x + y + 1.0; };
00066
00067         // Forcing term: f0 = -Laplacian(u) = -2(x+y)
00068         data.f0 = [](double x, double y) { return -2.0 * (x + y); };
00069
00070         // Neumann boundaries: du/dn = grad(u) * n
00071         data.f1 = [](double x, double y) { return -(x*x + 1.0); };                         // Bottom (n =
      0, -1) -> -uy
00072         data.f2 = [](double x, double y) { return y*y + 2.0*y + 1.0; };                    // Right  (n =
      1,  0) ->  ux
00073         data.f3 = [](double x, double y) { return x*x + 2.0*x + 1.0; };                    // Top    (n =
      0,  1) ->  uy
00074         data.f4 = [](double x, double y) { return -(y*y + 1.0); };                         // Left   (n =
      -1,  0) -> -ux
00075
00076
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::NEUMANN, ExecutionMode::SEQUENTIAL, funcType>
      solver(data);
00077         solver.build_mesh();
00078         solver.solve();
00079         solver.print();
00080         solver.export_to_vtk("test_17_neumann_sequential.vtk");
00081     }
00082
00083     // ======================================================================
00084     // TEST 18: ROBIN SEQUENTIAL (Jacobi) - Non-homogeneous Trigonometric
00085     // ======================================================================
00091     inline void run_test_18_robin_sequential(int mpi_rank) {
00092         if (mpi_rank == 0) {
00093             std::cout « "\n=== Running Test 18: ROBIN SEQUENTIAL (Jacobi) ===" « std::endl;
00094         }
00095
00096         auto data = create_default_data();
00097
00098         // Exact solution: u(x,y) = sin(x+y) + 2x + 3y
00099         data.u_exact_lambda = [](double x, double y) { return std::sin(x + y) + 2.0*x + 3.0*y; };
00100
00101         // Forcing term: f0 = -Laplacian(u) = 2*sin(x+y)
00102         data.f0 = [](double x, double y) { return 2.0 * std::sin(x + y); };
00103
00104         // Robin boundaries: u + du/dn = g
00105         // Note: ux = cos(x+y) + 2, uy = cos(x+y) + 3
00106         data.f1 = [](double x, double y) { return std::sin(x) - std::cos(x) + 2.0*x - 3.0; };
      // Bottom: u - uy
00107         data.f2 = [](double x, double y) { return std::sin(1.0+y) + std::cos(1.0+y) + 3.0*y + 4.0; };
      // Right:  u + ux
00108         data.f3 = [](double x, double y) { return std::sin(x+1.0) + std::cos(x+1.0) + 2.0*x + 6.0; };
      // Top:    u + uy
00109         data.f4 = [](double x, double y) { return std::sin(y) - std::cos(y) + 3.0*y - 2.0; };
      // Left:   u - ux
00110
00111
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::ROBIN, ExecutionMode::SEQUENTIAL, funcType>
      solver(data);
00112         solver.build_mesh();
00113         solver.solve();
00114         solver.print();
00115         solver.export_to_vtk("test_18_robin_sequential.vtk");
00116     }
00117
00118     // ======================================================================
00119     // TEST 19: DIRICHLET PARALLEL (Jacobi) - Non-homogeneous Polynomial
00120     // ======================================================================
00125     inline void run_test_19_dirichlet_parallel(int mpi_rank) {
```

```
00126         if (mpi_rank == 0) {
00127             std::cout « "\n=== Running Test 19: DIRICHLET PARALLEL (Jacobi) ===" « std::endl;
00128         }
00129 
00130         auto data = create_default_data();
00131 
00132         // Exact solution: u(x,y) = x^2*y + x*y^2 + x + y + 1
00133         data.u_exact_lambda = [](double x, double y) { return x*x*y + x*y*y + x + y + 1.0; };
00134 
00135         // Forcing term: f0 = -Laplacian(u) = -2(x+y)
00136         data.f0 = [](double x, double y) { return -2.0 * (x + y); };
00137 
00138         // Dirichlet boundaries: u evaluated exactly at the edges
00139         data.f1 = [](double x, double y) { return x + 1.0; };                           // Bottom
      (y=0)
00140         data.f2 = [](double x, double y) { return y*y + 2.0*y + 2.0; };                 // Right
      (x=1)
00141         data.f3 = [](double x, double y) { return x*x + 2.0*x + 2.0; };                 // Top
      (y=1)
00142         data.f4 = [](double x, double y) { return y + 1.0; };                           // Left
      (x=0)
00143 
00144 
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::DIRICHLET, ExecutionMode::PARALLEL, funcType>
      solver(data);
00145         solver.build_mesh();
00146         solver.solve();
00147         solver.print();
00148         solver.export_to_vtk("test_19_dirichlet_parallel.vtk");
00149     }
00150 
00151     // ============================================================================
00152     // TEST 20: NEUMANN PARALLEL (Jacobi) - Non-homogeneous Polynomial
00153     // ============================================================================
00158     inline void run_test_20_neumann_parallel(int mpi_rank) {
00159         if (mpi_rank == 0) {
00160             std::cout « "\n=== Running Test 20: NEUMANN PARALLEL (Jacobi) ===" « std::endl;
00161         }
00162 
00163         auto data = create_default_data();
00164 
00165         // Exact solution: u(x,y) = x^2*y + x*y^2 + x + y + 1
00166         data.u_exact_lambda = [](double x, double y) { return x*x*y + x*y*y + x + y + 1.0; };
00167 
00168         // Forcing term: f0 = -Laplacian(u) = -2(x+y)
00169         data.f0 = [](double x, double y) { return -2.0 * (x + y); };
00170 
00171         // Neumann boundaries: du/dn = grad(u) * n
00172         data.f1 = [](double x, double y) { return -(x*x + 1.0); };                      // Bottom (n =
      0, -1) -> -uy
00173         data.f2 = [](double x, double y) { return y*y + 2.0*y + 1.0; };                 // Right  (n =
      1,  0) ->  ux
00174         data.f3 = [](double x, double y) { return x*x + 2.0*x + 1.0; };                 // Top    (n =
      0,  1) ->  uy
00175         data.f4 = [](double x, double y) { return -(y*y + 1.0); };                      // Left   (n =
      -1,  0) -> -ux
00176 
00177 
      laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::NEUMANN, ExecutionMode::PARALLEL, funcType>
      solver(data);
00178         solver.build_mesh();
00179         solver.solve();
00180         solver.print();
00181         solver.export_to_vtk("test_20_neumann_parallel.vtk");
00182     }
00183 
00184     // ============================================================================
00185     // TEST 21: ROBIN PARALLEL (Jacobi) - Non-homogeneous Trigonometric
00186     // ============================================================================
00191     inline void run_test_21_robin_parallel(int mpi_rank) {
00192         if (mpi_rank == 0) {
00193             std::cout « "\n=== Running Test 21: ROBIN PARALLEL (Jacobi) ===" « std::endl;
00194         }
00195 
00196         auto data = create_default_data();
00197 
00198         // Exact solution: u(x,y) = sin(x+y) + 2x + 3y
00199         data.u_exact_lambda = [](double x, double y) { return std::sin(x + y) + 2.0*x + 3.0*y; };
00200 
00201         // Forcing term: f0 = -Laplacian(u) = 2*sin(x+y)
00202         data.f0 = [](double x, double y) { return 2.0 * std::sin(x + y); };
00203 
00204         // Robin boundaries: u + du/dn = g
00205         // Note: ux = cos(x+y) + 2, uy = cos(x+y) + 3
00206         data.f1 = [](double x, double y) { return std::sin(x) - std::cos(x) + 2.0*x - 3.0; };
      // Bottom: u - uy
00207         data.f2 = [](double x, double y) { return std::sin(1.0+y) + std::cos(1.0+y) + 3.0*y + 4.0; };
```

```
    // Right:  u + ux
00208         data.f3 = [](double x, double y) { return std::sin(x+1.0) + std::cos(x+1.0) + 2.0*x + 6.0; };
    // Top:    u + uy
00209         data.f4 = [](double x, double y) { return std::sin(y) - std::cos(y) + 3.0*y - 2.0; };
    // Left:   u - ux
00210
00211
    laplacian_solvers::Laplacian_Solver<SolverType::JACOBI, BoundaryCondition::ROBIN, ExecutionMode::PARALLEL, funcType>
    solver(data);
00212         solver.build_mesh();
00213         solver.solve();
00214         solver.print();
00215         solver.export_to_vtk("test_21_robin_parallel.vtk");
00216     }
00217
00218     // ========================================================================
00219     // TEST 22: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz)
00220     // ========================================================================
00225     inline void run_test_22_neumann_sequential_vs_parallel_schwarz(int mpi_rank) {
00226         if (mpi_rank == 0) {
00227             std::cout « "\n=== Running Test 22: NEUMANN SEQUENTIAL vs PARALLEL (Schwarz) ===" «
    std::endl;
00228         }
00229
00230         auto data = create_default_data();
00231
00232         // Exact solution: u(x,y) = x^2*y + x*y^2 + x + y + 1
00233         data.u_exact_lambda = [](double x, double y) { return x*x*y + x*y*y + x + y + 1.0; };
00234
00235         // Forcing term: f0 = -Laplacian(u) = -2(x+y)
00236         data.f0 = [](double x, double y) { return -2.0 * (x + y); };
00237
00238         // Neumann boundaries: du/dn = grad(u) * n
00239         data.f1 = [](double x, double y) { return -(x*x + 1.0); };                    // Bottom (n =
    0, -1) -> -uy
00240         data.f2 = [](double x, double y) { return y*y + 2.0*y + 1.0; };               // Right  (n =
    1,  0) ->  ux
00241         data.f3 = [](double x, double y) { return x*x + 2.0*x + 1.0; };               // Top    (n =
    0,  1) ->  uy
00242         data.f4 = [](double x, double y) { return -(y*y + 1.0); };
00243
00244         laplacian_solvers::Convergence_Test<SolverType::SCHWARZ, BoundaryCondition::NEUMANN, funcType>
    convergence_test(data);
00245         auto convergence_results = convergence_test.run_convergence_test();
00246         convergence_test.print_convergence_results();
00247     }
00248
00249 } // namespace laplacian_solvers
00250
00251 #endif // LAPLACIAN_SOLVERS_TEST_2_HPP
```

8.23 main.cpp File Reference

```
#include <mpi.h>
#include "laplacian_solvers_test_1.hpp"
#include "laplacian_solvers_test_2.hpp"
```

Include dependency graph for main.cpp:

Functions

- int main (int argc, char *argv[])

8.23.1 Function Documentation

8.23.1.1 main()

```
int main (
            int argc,
            char * argv[] )
```

Here is the call graph for this function:

8.24 README.md File Reference

8.25 results.md File Reference

Index